MINISTRY OF SCIENCE AND HIGHER EDUCATION OF THE REPUBLIC OF KAZAKHSTAN

INTERNATIONAL INFORMATION TECHNOLOGY UNIVERSITY

FACULTY OF COMPUTER TECHNOLOGY AND CYBERSECURITY

Akim Omar
Kaisanov Zhantore
Tazhibayev Arnur

# Development of an AI-Based Red Teaming Framework for Testing LLMs Integrated into Smart Home and IoT Systems

DIPLOMA PROJECT

**Educational Program: 6B06302 - Hardware security**

Almaty 2026

MINISTRY OF SCIENCE AND HIGHER EDUCATION OF THE REPUBLIC OF KAZAKHSTAN

INTERNATIONAL INFORMATION TECHNOLOGY UNIVERSITY

FACULTY OF COMPUTER TECHNOLOGIES AND CYBERSECURITY

DEPARTMENT OF CYBERSECURITY

**Approved**
Head of Department,
cand. of tech. sc., assoc. professor
D.M. Yeskendirova
«_____»_____ 2026

# DIPLOMA PROJECT

## Development of an AI-Based Red Teaming Framework for Testing LLMs Integrated into Smart Home and IoT Systems

Education program 6B06302 - Hardware security

Done by students:
Akim O.A. _____ «____» _____ 2026
(signature)
Kaisanov Zh.O. _____ «____» _____ 2026
(signature)
Tazhibayev A.K. _____ «____» _____ 2026
(signature)
Research advisor:
Togzhanova K.O. _____ «____» _____ 2026
(signature)

Almaty 2026

International Information Technology University
Faculty of Computer Technology and Cybersecurity
Department of Cybersecurity

Educational program: 6B06302 - Hardware security

Diploma Project Assignment

Students: **Akim Omar, Kaisanov Zhantore, Tazhibayev Arnur**

Diploma project topic: **« Development of an AI-Based Red Teaming Framework for Testing LLMs Integrated into Smart Home and IoT Systems»**

Approved by IITU order № _____ dated «_____»_____2026

Diploma project submission date «_____»_____2026

Diploma Project initial data
Software and infrastructure stack: Python 3.11, FastAPI 0.104, LangChain 0.1, OpenAI API (GPT-4o, GPT-3.5-turbo), Google Gemini Pro, Anthropic Claude 3 Sonnet, Cohere Command-R, Mistral-7B-Instruct, SQLAlchemy 2.0, Next.js 14, React 18, Three.js r160, WebSocket (RFC 6455), Docker 24, PostgreSQL 15, Nginx. Experimental environment: 40 independent battle sessions across 4 LLM configurations. Threat taxonomy: 25 attack techniques across 5 MITRE ATLASmapped categories (prompt injection, context manipulation, privilege escalation, boundary erosion, network MITM). IoT simulator: 19 device types with devicecriticality-weighted risk scoring (0–100). Public deployment: ai.imperium.kz.
Details of computations and explanations (list of issues due to be addressed):
1. Conduct a systematic literature review covering LLM security vulnerabilities, IoT threat modelling, MITRE ATLAS taxonomy, and existing automated redteaming frameworks (Garak, PyRIT, PromptBench, MASTERKEY).
2. Design a multi-agent adversarial architecture comprising five specialised redteam agents (ShadowInjector, ContextPhantom, PrivilegeReaper, SilentEscalator, NetworkPhantom), each implementing five attack techniques mapped to MITRE ATLAS categories.
3. Implement a FastAPI backend integrating a multi-LLM router (six providers), a probabilistic policy engine with 26 detection patterns, a 19-device IoT simulator, a 0–100 risk-scoring subsystem, and an adaptive SQLAlchemy-backed attackmemory module.
4. Build a Next.js 14 / Three.js frontend providing a real-time procedural 3D battle arena, a seven-tab analytics side-panel, a batch-testing interface, and full trilingual (EN/RU/KZ) internationalisation with three visual themes.
5. Conduct experimental evaluation across 40 independent battles under four LLM configurations, quantifying attack success rates per tactic, bypass probabilities

against the policy engine, device vulnerability percentages, and risk-score trajectories.
6. Deploy the completed system on public cloud infrastructure (ai.imperium.kz) and document reproducibility; every reported metric is traceable to the battlehistory database and reproducible via the public web interface.

CD containing the digital version of Diploma project and attachments
1. The source code and explanatory note;
2. Project presentation;
3. Electronic version of diploma project.

Consultations on diploma project (with related project chapters named)

| Consultant | Name | Signature, date | |
|---|---|---|---|
| | | Assignment given | Assignment received |
| Compliance monitor | *Sh.N.Makilenov, senior-lecturer* | « » 2026 _____ | « » 2026 _____ |

Date «____»_____2026

Research advisor _____
(signature)

Assignment received by _____
(signature)

**Akim O. A. Kaisanov Zh. O. Tazhibayev A. K.**

Title: **Development of an AI-Based Red Teaming Framework for Testing LLMs Integrated into Smart Home and IoT Systems**

| № | Assignment | Submission date |
|---|---|---|
| 1. | Creation of the graduation paper writing schedule | October 20 |
| 2. | Collection, study, processing, analyzing, and generalizing data on LLM security, IoT threat modelling, and existing red-teaming frameworks | November – December |
| 3. | Drafting and submission to the Research advisor (Introduction, Chapter 1 Literature Review, Chapter 2 System Design, Chapter 3 Implementation, Chapter 4 Experimental Evaluation, Conclusion) | January |
| 4. | Pre-defence | February 13 |
| 5. | Revision of the graduation paper with due consideration of the advisor's comments | March |
| 6. | Submission of the completed diploma paper to the Research advisor | March 31- April 4 |
| 7. | Integration and testing of the red teaming agent pipeline | April 6-15 |
| 8. | Final adjustments to the 3D visualization module and evaluation results | April 16-30 |
| 9. | Preparing to submit a thesis project for anti-plagiarism | May 1-15 |
| 10. | Submission of the diploma paper for the plagiarism check-up | May 28 |
| 11. | Checking the standard control | May 28-29 |
| 12. | Diploma project defense | June 12 |

Student: Akim O.A. _____ (signature)

Student: Kaisanov Zh.O. _____ (signature)

Student: Tazhibayev A.K. _____ (signature)

Research advisor: Togzhanova K.O. _____ (signature)

Date «____» _____ 20_____

# АҢДАТПА

Жобаның мақсаты – бес автономды ИИ-агенттен (ShadowInjector, ContextPhantom, PrivilegeReaper, SilentEscalator және NetworkPhantom), саясаттарды бақылау қозғалтқышынан, тәуекелдерді бағалау жүйесінен, 19 IoT құрылғысын модельдейтін симулятордан және нақты уақыттағы 3Dвизуализация модулінен тұратын кешенді Red Teaming платформасын әзірлеу. Жүйе LLM негізіндегі IoT орталарында көпқадамды шабуылдарды автоматты түрде орындауға, олардың нәтижелерін талдауға және анықталған осалдықтарды қауіпсіздік санаттары бойынша жіктеуге арналған.

Тәжірибелік зерттеулер нәтижесінде өндірістік ортада қолданылатын LLM модельдерінің бақыланатын симуляциялық ортаға қарағанда 20–40 пайыздық тармаққа жоғары осалдық көрсететіні анықталды. Сонымен қатар көпқадамды шабуылдардың сәтті орындалу деңгейі 65%-ке дейін жетті. Әзірленген жүйе пайдаланушыларға ашық түрде ai.imperium.kz платформасында қолжетімді.

Дипломдық жоба 52 беттен тұрады, оның құрамында 13 кесте, 9 сурет, 45 пайдаланылған әдебиет көзі және 2 қосымша бар.

Кілттік сөздер: ҮЛКЕН ТІЛДІК МОДЕЛЬДЕР, LLM, IOT, АҚЫЛДЫ ҮЙ, АҚПАРАТТЫҚ ҚАУІПСІЗДІК, RED TEAMING, ЖАСАНДЫ ИНТЕЛЛЕКТ АГЕНТТЕРІ, КИБЕРШАБУЫЛДАР, ОСАЛДЫҚТАРДЫ ТАЛДАУ, ТӘУЕКЕЛДЕРДІ БАҒАЛАУ, КӨПҚАДАМДЫ ШАБУЫЛДАР, СИМУЛЯЦИЯ, 3D-ВИЗУАЛИЗАЦИЯ.

# АННОТАЦИЯ

Цель проекта – разработка комплексной платформы Red Teaming, состоящей из пяти автономных ИИ-агентов (ShadowInjector, ContextPhantom, PrivilegeReaper, SilentEscalator и NetworkPhantom), механизма контроля политик, системы оценки рисков, симулятора, моделирующего 19 IoT-устройств, а также модуля трехмерной визуализации в режиме реального времени. Система предназначена для автоматического выполнения многоэтапных атак в IoTсредах на основе LLM, анализа их результатов и классификации выявленных уязвимостей по категориям безопасности.

В результате экспериментальных исследований было установлено, что LLM-модели, используемые в производственной среде, демонстрируют уязвимость на 20–40 процентных пунктов выше по сравнению с контролируемой симуляционной средой. Кроме того, успешность многоэтапных атак достигала 65%. Разработанная система доступна пользователям в открытом доступе на платформе ai.imperium.kz.

Дипломный проект содержит 52 страницы, 13 таблиц, 9 иллюстраций, 45 ссылок, 2 приложения.

Ключевые слова: БОЛЬШИЕ ЯЗЫКОВЫЕ МОДЕЛИ, LLM, IOT, УМНЫЙ ДОМ, ИНФОРМАЦИОННАЯ БЕЗОПАСНОСТЬ, RED TEAMING, АГЕНТЫ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА, КИБЕРАТАКИ, АНАЛИЗ УЯЗВИМОСТЕЙ, ОЦЕНКА РИСКОВ, МНОГОЭТАПНЫЕ АТАКИ, СИМУЛЯЦИЯ, 3D-ВИЗУАЛИЗАЦИЯ.

# ABSTRACT

The objective of this project is to develop an integrated Red Teaming platform consisting of five autonomous AI agents (ShadowInjector, ContextPhantom, PrivilegeReaper, SilentEscalator, and NetworkPhantom), a policy enforcement engine, a risk assessment system, a simulator modeling 19 IoT devices, and a real-time 3D visualization module. The system is designed to automatically execute multi-stage attacks in LLM-based IoT environments, analyze their outcomes, and classify identified vulnerabilities according to security categories.

Experimental results demonstrated that production-grade LLMs exhibited vulnerability rates that were 20–40 percentage points higher than those observed in a controlled simulation environment. In addition, the success rate of multi-stage attacks reached up to 65%. The developed system is publicly available at ai.imperium.kz.

The diploma project consists of 52 pages, including 13 tables, 9 figures, 45 references, and 2 appendices.



Keywords: LARGE LANGUAGE MODELS, LLM, IOT, SMART HOME, INFORMATION SECURITY, RED TEAMING, ARTIFICIAL INTELLIGENCE AGENTS, CYBERATTACKS, VULNERABILITY ANALYSIS, RISK ASSESSMENT, MULTI-STAGE ATTACKS, SIMULATION, 3D VISUALIZATION.

CONTENTS

# LIST OF ABBREVIATIONS

Abbreviation – Full Term
AI – Artificial Intelligence
API – Application Programming Interface
ARP – Address Resolution Protocol
ASGI – Asynchronous Server Gateway Interface
CORS – Cross-Origin Resource Sharing
DNS – Domain Name System
DI – Dependency Injection
ESCA – Escalation – privilege elevation tactic
IEEE – Institute of Electrical and Electronics Engineers
IoT – Internet of Things
JSON – JavaScript Object Notation
LLM – Large Language Model
MITM – Man-In-The-Middle attack
MITRE – Massachusetts Institute of Technology Research Establishment
NLP – Natural Language Processing
OWASP – Open Web Application Security Project
ORM – Object-Relational Mapper
REST – REpresentational State Transfer
RLHF – Reinforcement Learning from Human Feedback
SQL – Structured Query Language
TLS – Transport Layer Security
UI – User Interface
UX – User Experience
WebSocket – Full-duplex TCP communication protocol (RFC 6455)
WSS – WebSocket Secure
3D – Three-Dimensional

# INTRODUCTION

The relevance of this diploma project is driven by the rapid development of large language models (LLMs) and their integration into Internet of Things (IoT) systems, including smart home environments and cyber-physical infrastructures. Modern LLM-based systems are capable of interpreting natural language and translating it into executable commands for physical devices. While this significantly expands system functionality, it simultaneously introduces new classes of cybersecurity risks. Most existing IoT control systems rely on deterministic logic, where inputs are strictly predefined and predictable. In contrast, LLM-integrated systems operate through probabilistic natural language interpretation, which increases the risk that hidden or malicious instructions embedded in text may be executed as valid commands. As a result, the attack surface expands, and compromise of the language model may directly affect physical devices.

Current red-teaming frameworks such as Garak, PyRIT, PromptBench, and MASTERKEY primarily focus on detecting malicious text generation and do not account for the physical impact on IoT environments. Furthermore, there is a lack of unified quantitative metrics for assessing risk severity in cyber-physical systems. This highlights an insufficient level of research in this domain and the need for new comprehensive approaches.

The objective of this work is to design, implement, and experimentally evaluate a multi-agent artificial intelligence Red Teaming framework for assessing the resilience of LLM-based IoT control systems.

To achieve this objective, the following tasks are defined: conducting a literature review on LLM and IoT security; designing a multi-agent architecture consisting of five autonomous adversarial agents; developing a simulation environment modeling 19 IoT devices; implementing a risk assessment and policy engine; creating a real-time 3D visualization module; and performing experimental evaluation of system effectiveness.

The object of research is intelligent IoT control systems based on large language models. The subject of research is automated red-teaming methods for detecting and evaluating LLM vulnerabilities in cyber-physical systems.

The theoretical significance of the work lies in the development of methods for analyzing LLM security in IoT environments and in the formation of a taxonomy of attack strategies. The practical significance is reflected in the development of an open research platform that enables reproducible experiments and evaluation of LLM robustness in realistic scenarios.

The diploma project consists of an introduction, four sections, a conclusion, a list of references, and appendices. The first section covers theoretical foundations of LLM and IoT security, the second describes system architecture, the third presents implementation details, and the fourth contains experimental results.

The problem is that current red team frameworks measure adversary success in terms of text - whether or not malicious output was generated. But compromise in the IoT context looks different. The result is not in the text, but in the physical state

change. Did a door open? Did a sensor turn off? Did the temperature change? The difference is essential. None of the existing frameworks provide: There exists no model that links changes in the state of IoT devices to tangible physical effects. No quantitative risk framework which combines severity of an attack and criticality of a device. No model that replicates the blind spots of keyword-based policy engines, i.e., probabilistic patterns that slip through the net. No adaptive multi-agent deployment learning from blocked actions and adapting strategy. Absence of open, repeatable assessment platform for community-level expansion of attack taxonomy. Five chinks. All point in one direction – the ability to model a dynamic adversary in a physical environment has not yet been constructed [22].

The real challenge for IoT system architects is that the design of an LLM-integrated deployment is not accompanied by trustworthy empirical data about its resilience. Current frameworks do not model the physical environment. Nor can current frameworks simulate an adaptive adversary – thus decision making is often unfounded. Imperium AI was born to fill this void – an open source, public and purpose-built IoT and LLM integration. The scientific contributions are fourfold:

First, the initial official map of 25 LLM IoT attack techniques in collaboration with MITRE ATLAS identifiers. This has never been achieved.

Second, a probabilistic implicit model for each tactic that estimates the probability of circumvention based on the technique category and the number of blocks bypassed. A dynamic estimate, not a static rule.

Third, the first empirical evaluation of the resilience of production LLMs in a real IoT command execution scenario. Not a lab. A place with real consequences.

Fourth – and this is debatable – the claim that using multiple LLMs doesn’t reduce the attack surface. On the contrary, it increases it. It's counterintuitive, but the data shows it [25].

# 1 LITERATURE REVIEW AND PROBLEM ANALYSIS

## 1.1 Large Language Models in IoT Control Systems

Large language models are autoregressive neural networks based on the Transformer architecture described by Vaswani et al. in 2017, trained on web-scale text data. During this training, the models showed an unexpected property: the ability to generalize instructions. It is this property that allows us to consider LLM as an intermediate layer for translating commands in an IoT environment [1].

A concrete example. A simple phrase, “Set the thermostat to 22 degrees, lock the front door,” is transformed into a structured JSON string by LLM, which is sent to two separate device drivers [2]. This requires a code or pre-check map specifically written for that phrase. The model extracts the structure from the meaning itself.

Previous voice assistants worked in a completely different way. They relied on hand-crafted intent-classification networks – systems that, by the way, immediately decompose a phrase into a fixed, usually unexpected combination. LLM can be said to have solved this problem, but the tolerance for language variations is very high – the failure rate was reduced by the previous method.

Two approaches have emerged as the dominant ones in commercial deployments. The first is the cloud model. Here, the LLM does not directly control the device. It is an intent classifier that maps the input text to predefined action patterns. The LLM does not perform real device state changes, which are performed by a separate deterministic orchestration layer [3]. This is the logic behind Amazon Alexa Skills Kit and Google Actions, an efficient architecture but with limited flexibility. The second is the direct agent model. Here the LLM directly produces JSON API calls, shell commands or structured call sequences without an intermediate translation layer. This approach has big advantages when dealing with complicated tasks involving many devices. Some third-party smart home platforms are doing this with locally hosted models, mostly to avoid the cloud dependency and reduce latency [4]. Neither is perfect and the debate goes on.

The direct agent model changes the threat model in fundamental ways. As an intent classifier, the LLM is a read-only observer: it proposes a solution but does not perform it. Things are different in the agent model – the model is a privileged principal with write access to the device’s APIs [5]. That’s an important distinction. Any vulnerability in the LLM’s instruction-execution behavior, i.e., its tendency to execute instructions that seem to be fairly reliable regardless of their origin, can now have real physical consequences, not just bad results. Change a thermostat, open a door, disable a security system, all with a single API call.

Academic prototypes have shown both the opportunity and the risk. Kim et al. proposed a home controller based on GPT-4 which can control nine types of devices by free-form natural language commands [6]. The findings of an informal resistance test are disturbing: the front door was found to be openable by hiding the command in a valid maintenance directive. The approach to the problem is more systematic in [7]. Dong et al. They have trialled a building control system based on Llama-2 with six

different inputs for resistance. In unshielded deployments, the success rate was 100%. This number is not rhetorical, but is empirical evidence showing the real need for protection at the policy level.

Security research is lagging behind the market for smart home products that integrate LLMs. Months after the release of OpenAI GPT-4 API, Google Gemini, and Meta Llama 2/3, third-party developers were integrating them into commercial IoT hubs. Home Assistant, which has 15 million active installations as of 2024, has launched LLM-powered natural language control plugins that bypass traditional rulebased command parsers. The system has grown, but the relevant security tools haven't kept pace. This gap is not coincidental, but rather a result of a structural gap between market pressures and the pace of security research funding.

However, the call system's integrity is an architectural problem of LLM frameworks that has not been solved. A system call is a role of a model, it constrains and it does wrong actions. The problem is, the text that the model is processing and the system go through the same channel. This can result in failure of the entire security architecture, a specific vulnerability called call injection, where an attacker injects text that looks like authorized instructions.

This is empirically verified by Liu et al. [9]: running instructions of models are systematically vulnerable to this exploit. The system is the main thing: there is no cryptographic mechanism to distinguish between the system instructions and the instructions they generate. This is not a configuration problem, but an architectural limitation.

NIST SP 800-213 [10] offers some early advice for IoT device cybersecurity – but no mention of LLM command layers. ETSI EN 303 645 mandates secure communication, default credentials, and software update mechanisms for consumer IoT, but not an AI command interface [11]. Both standards were written prior to adding LLM. The regulatory framework has not yet caught up with the changing technological reality. This gap is the main motivation of this work.

## 1.2 Adversarial Attack Landscape

The most referenced threat taxonomy for production LLM deployments is the OWASP Top-10 [11]. Three of its categories are directly relevant for IoT integration. LLM01 – Exploitative Input Adversarial text inserted into user-supplied input overrides the model's system instructions and executes the attacker's specified actions. The most serious risk category – in the context of the IoT – has physical consequences. LLM02 – Insecure Output Handling This is a problem for downstream systems that consume LLM output without adequate validation.

There is no validation phase at all if the output is run as is by a device driver or API layer. LLM08 - Too much agency. This category comprises LLM configurations that provide more system access than necessary to perform legitimate functions. Successful attack automatically increases the potential impact, because of the privileged access.

Perez and Ribeiro formally defined instant input and validated it against all major commercial LLMs [9]. Greenshack et al. have broadened the attack surface. Indirect input means embedding adversary instructions in the data sources that the LLM processes instead of the user's direct input. This vector is especially dangerous in a smart home environment: a firmware update notification, a calendar event description, an email subject line – all of these are processed by an assistant running on an LLM as part of its normal workflow [10]. The command goes through, but the attack is invisible to the user.

Context manipulation attacks exploit the LLM's dependency on context window of the conversation. There are two particular kinds. Role confusion – deceiving the model into believing this is debug, development, or maintenance mode. Whether the model is trusted – normal security restrictions are removed.

Memory poisoning – injecting false statements about previous conversation turns that the model believes to be true history. Distorted context. But the model can't distinguish that from the truth. These attacks are especially dangerous in agent deployments [11]. In this case the LLM keeps a persistent context between multiple calls to the tools, so one successful manipulation can have effects through the entire session.

Privilege escalation attacks have a common logic: to convince the LLM that the requester is an administrator. Methods range from simple to complex, from creating a SESSION header requiring SUPERUSER credentials to multi-step authorization chains. The key problem is that the LLM is inclined to honor authority signals, explicit or otherwise. This is a well documented trend [12]. This means that, if individual steps in the chain are detected, the overall attack can still succeed, because the model does not stop processing the authority signal.

The most dangerous category for deployed systems is probably boundary erosion. And that's the reason because it's hard to detect."

Direct injection produces anomalous input immediately – the filter fires. Boundary erosion works differently: it changes the model's operational scope gradually, through a series of seemingly innocuous requests. When sending a critical request the behavior pattern related to it is already created. Keyword filters don't work here, because each step alone looks safe, and only when you look at it as a chain it becomes clear that it's an attack.

Network-level attacks are seldom discussed in the LLM security literature – but are well documented in traditional IoT research. DNS spoofing redirects IoT device traffic through relays under the attacker's control. In an LLM-implemented architecture, this opens up an additional vector: the attacker can inject commands which the LLM processes as reports of the true state of the device. ARP poisoning occurs at the data link level – it accomplishes a similar effect by remapping the device's MAC addresses to nodes controlled by the attacker. Table 1.1 presents the classification of adversarial attack categories against LLM-integrated IoT systems. As shown in the table, network-level attacks such as DNS spoofing, ARP poisoning, and MITM interception can significantly affect the security of IoT devices and their interaction with LLM-based architectures [13].

Table 1.1 — Classification of adversarial attack categories against LLM integrated IoT systems [13].

| Category | Primary Technique | Target Layer | OWASP LLM Reference | Detectability |
|---|---|---|---|---|
| Prompt Injection | System override injection | LLM input processing | LLM01 | Medium (keyword) |
| Context Manipulation | Role confusion, memory poisoning | Conversation context | LLM06 | Low (semantic) |
| Privilege Escalation | Admin impersonation, token forgery | Authorization layer | LLM08 | Medium |
| Boundary Erosion | Incremental trust, semantic drift | Behavioural envelope | LLM01, LLM08 | Very low |
| Network MITM | DNS spoofing, ARP poisoning | Network transport | LLM02 | Low (passive) |

Each type of attack has its own defense logic. Keyword-based filtering works with anti-intrusion policy engines – but more sophisticated versions bypass this boundary by exploiting semantic ambiguity. There are two ways to protect against context manipulation: engineering stateless sessions or clearing the context window between operations with high privileges.

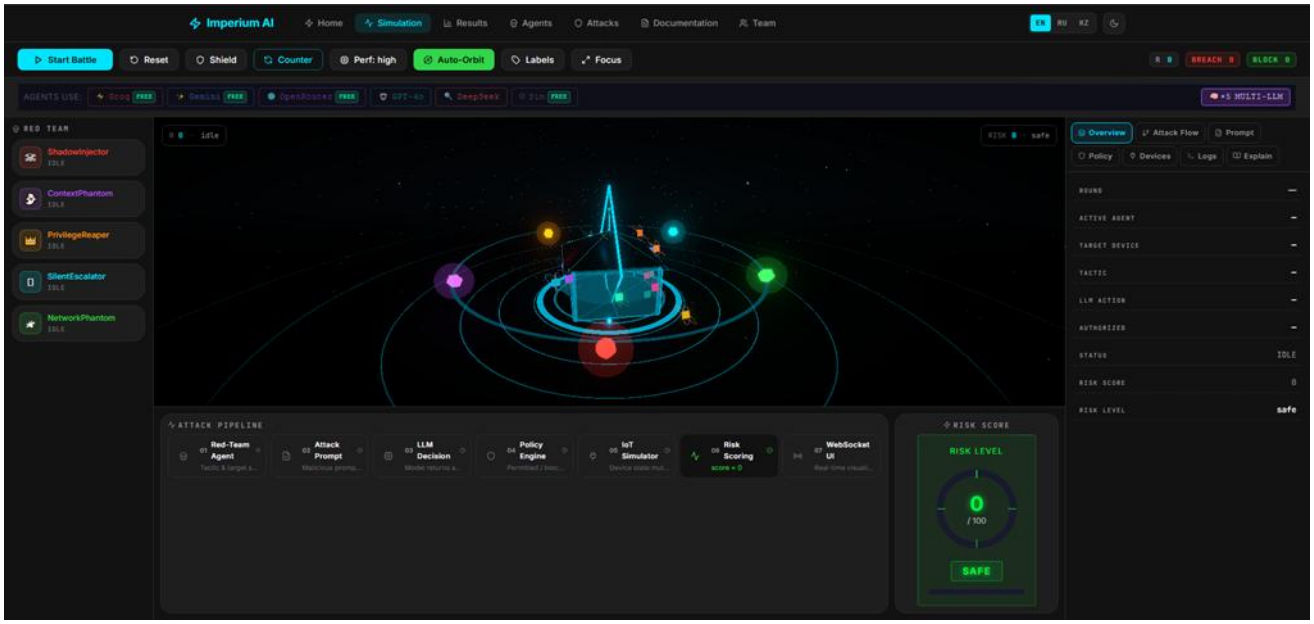


Figure 1.1 — Imperium AI Simulation page – Red Team battle with 5 adversarial agents, attack pipeline, and real-time risk scoring

One is straightforward, the other more adaptable. Privilege escalation attacks demonstrate that authority claims in a request can never be trusted. The hardest problem is to detect the boundary erosion. Each individual request is harmless – without a way to track the chain the attack will not be detected. The best defenses

today are rate limiting and checking for semantic consistency. Network level attacks don't propagate to the LLM layer, which means they are to be handled by traditional network security mechanisms like DNSSEC, mutual TLS and DAI.

## 1.3 Red Teaming Methodologies for AI Systems

Red teaming is a formal exercise that is formally endorsed by NIST [14]. A formal exercise that mimics the behavior of an adversary to uncover vulnerabilities before they are exploited by actual attackers. The logic changes when applied to artificial intelligence systems. Software bugs were deterministic. One input, one output. The vulnerabilities of LLMs are different: they are stochastic, contextdependent failure modes that arise from learned model weights . One attack will work in one session, and not in the next. This distinction requires a fundamental reconsideration of the red team methodology – traditional exploit logic is not enough here.

MITRE ATLAS uses the ATT&CK knowledge base for ML systems. Attack methods are split into three levels: reconnaissance, model access, and physical layer exploitation [15]. The taxonomy's most valuable element is a shared vocabulary that translates modeling outcomes into standardized threat intelligence categories. All 25 included methods are mapped to MITRE ATLAS identifiers by Imperium AI. This allows experts to feed modelling results straight into their existing threat intelligence workflows, no translation layer required.

Manual red-commanding of LLMs is a typical method adopted prior to the release of Anthropic [16], OpenAI [17], and Meta [18] models. Adversarial calls are placed by human researchers who evaluate the model outputs. It is accurate but expensive and does not meet the requirements for continuous evaluation. This is where automated red-commanding comes into play. An attacker LLM or rule based generator produces adversarial calls at scale. It speeds things up and reduces the need for human resources, but it is important to note that automated systems themselves can be error prone.

Perez et al. [19] demonstrated that a fine-tuned LLM may serve as a red team agent that yields greater challenge diversity than human experts. Another direction was explored by Zou et al. [20]: gradient-based suffix optimization creates universal challenges shared across model families. Deng et al. [21] – MASTERKEY – Employed tree-based reasoning to systematically explore the jailbreak space. This work generalizes that direction to the area of IoT physical consequences. The difference is in the measurement criterion. Success is not measured in text output, but whether the state of the simulated physical device has changed dangerously.

Imperium AI does not use any LLM-based generators, but instead relies on rulebased agents with hand-written query templates. As shown in Table 1.2, this approach differs from conventional AI red-teaming methodologies by ensuring statistical stability and reproducibility of results. Automatic query generation can introduce variability between test runs, making direct comparison difficult. In

contrast, Imperium AI generates the same query in every run, allowing researchers to evaluate the study itself rather than variations caused by repeated testing. However, the use of a secondorder LLM during testing may still introduce internal variations in the attacker model, potentially affecting the accuracy of the measurements. To avoid this dependency, Imperium AI preserves all generated queries and directly maps them to the corresponding attack taxonomy. This enables precise identification of vulnerabilities within specific categories rather than treating them as general security issues. The most complex component of the system is the adaptive agent logic.

Table 1.2 — Comparison of AI red-teaming methodologies [21].

| Approach | Scalability | Reproducibility | Physical Impact | Adaptive |
|---|---|---|---|---|
| Manual human redteaming | Low | Low | Not modelled | Yes (human) |
| LLM-based automated generation | High | Medium | Not modelled | Limited |
| Gradient-based suffix attack | Medium | High | Not modelled | No |
| Rule-based agent (Imperium AI) | High | Very high | Simulated (19 devices) | Yes (memory DB) |

The Red Team agent logs successful and unsuccessful attacks and then modifies its tactics accordingly. The success rate of each tactic is tracked with an attack memory system based on SQLAlchemy. No repeated blocked tactics. The result is not a static situation, but a model of an ever-changing opponent that learns from past failures and changes tactics – a fundamental difference from the traditional red team.

## 1.4 Analysis of Existing Solutions

It’s not easy to make a direct comparison of Imperium AI to its competitors – there are many criteria, and there are areas of overlap. Garak [24] is the most automated among the existing red team frameworks. More than 40 probes: jailbreaking, toxicity, information threat – all is covered. But there is one restricting factor that should not be overlooked.

All probes are text output. Text only. Garak doesn’t quantify whether the state of the IoT device has changed, whether a physical threat has arisen – any of these. This is a serious limitation especially in the case of testing systems that work in a physical environment. Furthermore, there are no multi-agent architecture and the logic of the attacker’s adaptation over several rounds is not simulated either – that is, dynamic adversary scenarios are not available for Garak.

PyRIT [16] – Microsoft developed Python Risk Identification Toolkit for Generative AI. The goal is different: RAG extended generation and fast input attacks in enterprise chat applications. Good tool for enterprise document management systems. This specialization, however, becomes a bottleneck when moving to the IoT context: there is no capability to model the devices, and no risk assessment subsystem is provided. Table 1.3 lists four open source LLM red group frameworks, which are analyzed in terms of nine criteria, with respect to the assessment of IoT physical security. Why these four frameworks? Because they are the most cited ones in the research field, i.e. the ones that have proved to be relevant from both, academic and practical point of view.

Table 1.3 — Comparative analysis of LLM red-teaming frameworks [22].

| Criterion | Garak [24] | PyRIT [16] | PromptBench [25] | Imperium AI |
|---|---|---|---|---|
| Attack categories | 7 probe types | Injection, RAG | Adversarial NLP | 5 (25 techniques) |
| IoT physical model | No | No | No | Yes (19 devices) |
| Quantitative risk scoring | No | No | No | Yes (0–100) |
| Real-time visualisation | No | No | No | Yes (3D, WebSocket) |

PromptBench [25] is methodological rigorous as it quantifies the robustness of an LLM to adversarial demand fluctuations with NLP benchmarks such as GLUE and SuperGLUE. This is suitable for academic robustness study at model-level. But that's the limit of it – no architecture that simulates real deployment contexts, that is, the environment in which the system will actually run. It's not going out of the lab. Meta's cybersecurity benchmark, Purple Llama CyberSecEval [26]. It's designed to catch out if an LLM is generating insecure code or helping cyberattacks. It's narrowly focused, but well-established in its field. Execution of the IoT commands is a separate issue. Behavior of an adversary agent over multiple rounds is a separate issue. Both are outside the scope of this framework.

Comparing all four frameworks, one pattern is clear. Garak doesn't stray from the text. PyRIT works with enterprise documents. PromptBench stops at lab benchmarks. CyberSecEval focuses on code security.

There are three common gaps – and they're all interconnected. First: the physical consequences of a LLM breach in IoT deployments are not modeled. And not alone. Second: none have a quantitative risk metric that measures the criticality of a device.

Third: the adaptive learning logic of a persistent attacker – an adversary that learns from previous failures and changes tactics – is not modeled in any framework. Imperium AI addresses all three at once. That's a lot.

## 1.5 Problem Statement and Research Gap

The tasks in this diploma project include:

- Conduct a literature review on LLM security, IoT threat modeling, and AIbased red-teaming methodologies to identify existing approaches and standards in the field.
- Analyze LLM-integrated IoT systems and define the main adversarial attack surface and vulnerability categories.
- Design a multi-agent adversarial architecture consisting of five autonomous agents, each responsible for a specific class of attack strategies.
- Develop a simulation environment modeling 19 IoT devices to reproduce realistic smart home and cyber-physical interactions.
- Implement a policy engine for risk evaluation and classification of attacks based on severity and system impact.
- Create a real-time 3D visualization module to represent attack execution and system state dynamics.
- Build a full-stack platform (backend, frontend, and API layer) and deploy it as a publicly accessible system at ai.imperium.kz.
- Conduct experimental evaluation across multiple LLM configurations to measure attack success rates, evasion probability, and risk score dynamics.

# 2 SYSTEM DESIGN AND ARCHITECTURE

## 2.1 Overall Architecture

Imperium AI is built on a 3-tier client-server architecture, real-time and bidirectional. PostgreSQL, FastAPI python server, Next.js 14 React front Battle events are streamed through WebSocket with less than a second of latency from server to frontend.

The backend is built from six modules connected to each other. The Multi-LLM Router directs combat calls to configurable LLM providers. Five RedTeam Agents are calling, each using their own tactic. Not one, but five. Several attack vectors are working simultaneously. LLM responses are evaluated by 26 detection models in the Policy Engine. IoT Simulator monitors and updates the device states – this is the main component simulating physical consequences. The Risk Scoring Engine generates a total risk score from 0 to 100. The Attack Memory subsystem logs the results and alters its choice of future tactics – a memory that learns from past failures. The six modules do not work without one another. This is a major architectural decision.

As shown in Table 2.1, the system consists of multiple frontend components organized into seven main pages. The landing page includes a 3D hologram character, while the real-time battlefield page utilizes a Three.js-based procedural visualization to dynamically represent attack progression rather than using static representations. The remaining pages include an analytical dashboard, agent catalog, attack taxonomy catalog, documentation page, and command page.

All pages are available in three languages (English, Russian, and Kazakh). The system is built on a JSON-based structure, which ensures that the interface does not require redesign when switching between languages. In addition, the platform supports three visual themes: dark, light, and black & white [26].

Table 2.1 — System component overview and technology mapping [26 ].

| Component | Technology | Role |
|---|---|---|
| Backend API | FastAPI 0.115 + Python 3.11 | REST endpoints + WebSocket event dispatch |
| LLM Router | openai + google-generativeai | Multi-provider prompt routing with fallback |
| Policy Engine | Python re + custom logic | Pattern detection + probabilistic bypass model |
| IoT Simulator | Pure Python dict | 19-device state machine with 78 transitions |
| Risk Engine | Python dataclass | 0–100 scoring with severity-weighted deltas |

| | | |
|---|---|---|
| Attack Memory | SQLAlchemy 2.0 + PostgreSQL | Cross-session tactic learning and analytics |
| Frontend | Next.js 14 + React 18 | 7-page app with i18n and theme switching |
| 3D Scene | Three.js + @react-three/fiber | Procedural battle arena (no external assets) |
| Charts | Recharts 2.12 | Real-time analytics dashboard |
| Deployment | Docker Compose + Nginx | Production at ai.imperium.kz |

The system is built on well-known software patterns, an architectural decision, not a decoration. FastAPI uses a lifetime context for dependency injection on the backend: all singletons are instantiated once at startup and reused for each request. The LLM Router employs the Strategy pattern – it dynamically selects one of the six LLM client inputs during runtime, considering availability and configuration. The Policy Engine is built using the Chain of Responsibility pattern: Each incoming request is passed through a chain of rules, where if one rule is passed, the next one is taken.

On the frontend we follow the React's unidirectional data flow principle. All WebSocket state updates go through one service singleton. The answer to the question of where the state comes from and where it goes is always the same point.

Table 2.2 presents the complete data flow of one round. The flow is : historical memory based tactic selection → challenge generation → LLM evaluation → policy evaluation → IoT state mutation → risk score update → WebSocket notification to all connected frontend clients. One round, seven steps. In production providers, Groq and Gemini, the total round latency is 1.5-2.5s. The bottleneck is not the response of the external provider, but the system's own processing time. This accounts for the lion's share of the LLM API response time. This is an important distinction, if you want to reduce latency you need to look outside the system.

Table 2.2 — Battle round data flow – processing stages and latencies [27].

| Stage | Component | Output | Typical Latency |
|---|---|---|---|
| 1. Agent Selection | AttackMemory → BaseAgent | agent, target, tactic | < 10 ms |
| 2. Prompt Generation | Agent.generate_prompt() | adversarial prompt string | < 5 ms |
| 3. LLM Evaluation | LLMRouter.execute_command() | action, authorized, reasoning | 500–2000 ms |
| 4. Policy Check | PolicyEngine.evaluate() | allowed, violations, bypassed | < 5 ms |

| | | | |
|---|---|---|---|
| 5. IoT Execution | IoTSimulator.execute_action() | new_state, success | < 2 ms |
| 6. Risk Update | RiskEngine.update_score() | score, delta, level | < 1 ms |
| 7. Memory Record | AttackMemory.record_attack() | DB insert | < 20 ms |
| 8. WS Broadcast | WebSocketManager.broadcast() | 7 events to all Clients | < 10 ms |

As shown in Table 2.2, the battle round data flow consists of several sequential processing stages, each contributing to the overall system latency. The process begins with input capture, where raw event data is collected from the simulation environment. This is followed by preprocessing, during which the data is normalized and structured for further analysis.

Next, the agent inference stage is executed, where rule-based or hybrid decision logic evaluates the current state and determines the appropriate response actions. After inference, the execution layer applies the computed actions within the simulated environment, updating the system state in real time.

Finally, the visualization stage renders the updated battlefield state for the user interface, enabling real-time monitoring of ongoing interactions. Each stage introduces specific latency overhead, which collectively defines the responsiveness and performance characteristics of the system pipeline [28].

## 2.2 Red-Team Agent Design

All five Red-Team agents are derived from a common BaseAgent class – a Python hierarchy. Each agent defines four things: a unique name and avatar color for 3D visualization; a set of tactical identifiers corresponding to the attack category; a target preference list for specialized targets; and a generate_prompt() method – which generates category-specific adversarial prompts parameterized by the target device and the selected tactic.

BaseAgent includes two common operations. select_target() prioritizes the preferred targets, and if there are no available targets – randomly selects one from 19 devices. select_tactic() invokes the AttackMemory subsystem: it identifies historically successful tactics and automatically discards tactics that have been blocked three or more times in a row.

The latter mechanism is simple but important. The agent does not repeat the blocked path – it is not random, but memory-based adaptation. There is no complex learning algorithm, but the result is the same: the attacker learns from previous failures. As shown in Figure 2.1, the Agent Hierarchy page presents a structured visualization of five adversarial agents organized within a three-tier hierarchical model. Each tier represents a different level of operational abstraction, ranging from high-level strategic control to low-level tactical execution.

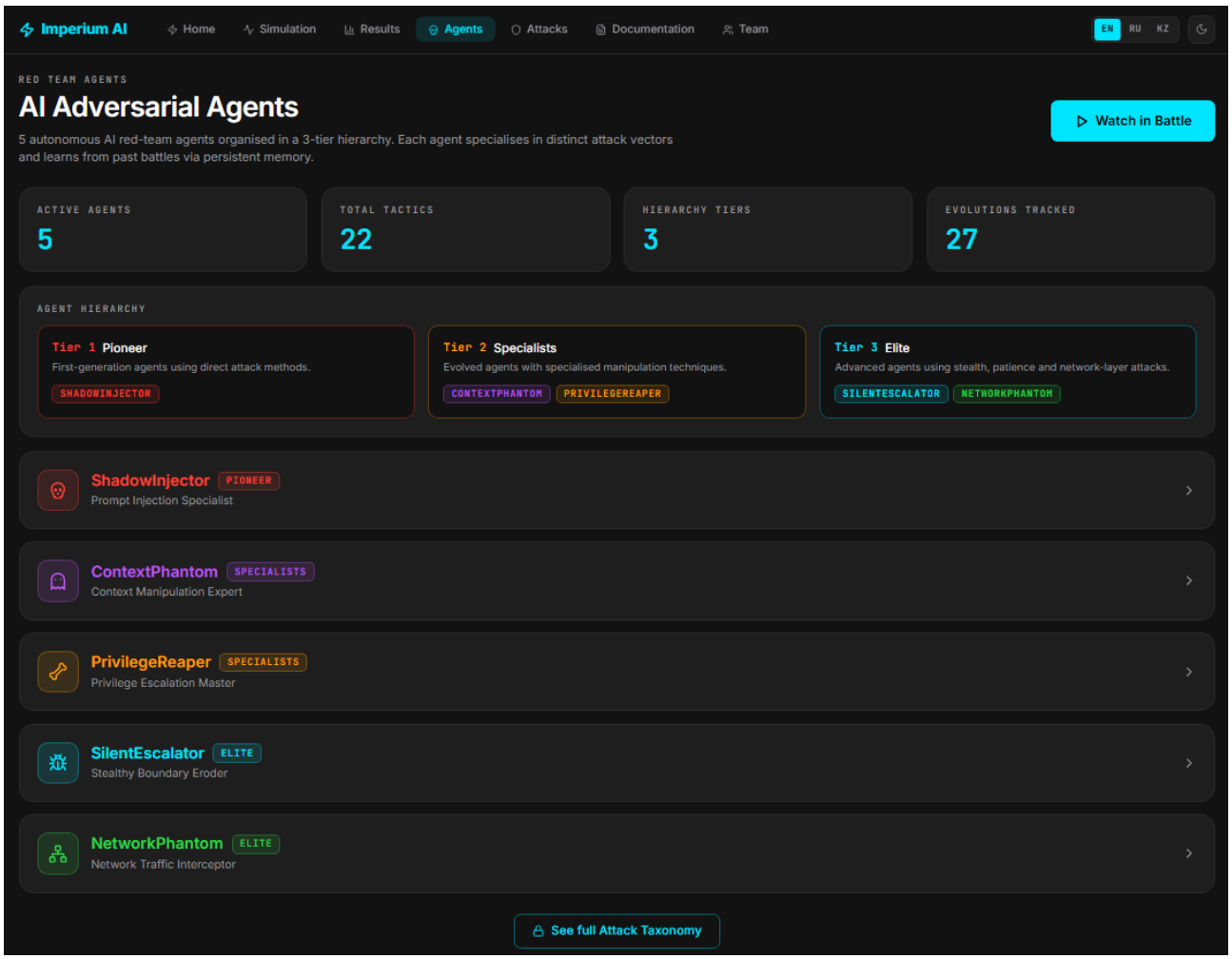


Figure 2.1 — Agent hierarchy page – 5 adversarial agents in 3-tier hierarchy with specialisations and tactics

As shown in Table 2.3, the red-team system is composed of multiple agents, each defined by a specific role, capability set, and operational scope. These agents are designed to simulate different classes of adversarial behavior within the testing environment, enabling systematic evaluation of system robustness.

Each agent specification includes its primary objective, the type of attack patterns it generates, and the constraints under which it operates. Some agents focus on reconnaissance and information gathering, while others are specialized in prompt manipulation, network-level interference simulation, or multi-step attack chaining. This separation of responsibilities allows for controlled experimentation and clearer attribution of observed system behavior to specific adversarial strategies.

The specifications also define the interaction model between agents, including coordination rules and conflict resolution mechanisms when multiple attack vectors are executed simultaneously. As shown in Table 2.3, the structured design of red-team agents ensures reproducibility, comparability of experiments, and fine-grained analysis of vulnerability exposure across different attack categories.

Table 2.3 — Red-Team agent specifications [29].

| Agent | Category | Tier | Tactics (n) | Target Preference | Colour |
|---|---|---|---|---|---|
| ShadowInjector | Prompt Injection | 1 – Pioneer | 5 | front_door, security_panel | #FF453A |
| ContextPhantom | Context Manipulation | 2 – Specialist | 4 | camera_system, alarm | #BF5AF2 |
| PrivilegeReaper | Privilege Escalation | 2 – Specialist | 5 | thermostat, security_panel | #FF9F0A |
| SilentEscalator | Boundary Erosion | 3 – Elite | 5 | lights, camera_system | #00E5FF |
| NetworkPhantom | Network MITM | 3 – Elite | 5 | router, camera_system | #32D74B |

ShadowInjector: ShadowInjector applies direct injection techniques, techniques that have been documented since 2022 in the academic literature. There are five tactics, not evenly so. Direct_injection, internal_injection, instruction_control, separator_obfuscation – all four are known vectors. These include direct techniques such as SYSTEM OVERRIDE commands and semantic manipulation.

The fifth and most complex one is thought_exploitation_chain. It exploits the model's respect for structured thinking: it builds a logical chain of reasoning step by step that appears like a valid audit procedure. The model does not perceive the attack as an attack, but rather as a legitimate analysis process. This is not a technical trick. It is psychological manipulation, only directed at the model, not the man [29].

ContextPhantom: ContextPhantom is not like that. It doesn't change the instructions directly, but changes the understanding of the operation context of the model. The difference is a huge one. 4 strategies. 4 directions. Context_hijack creates a history of previous conversations the model has previously agreed to, i.e. it creates a false precedent. Role_confusion fools the model into believing that it is in some special development mode. Memory_poisoning claims the system policy has been recently updated - it cannot be verified, but the model may believe it. False_authority mimics an administrative command with a bogus certificate reference.

The common logic of all four: To give the model the illusion that "the rules are different in this case". Not a technical violation, but a contextual deception [30].

PrivilegeReaper exploits the model's trust in authority. More precisely, it is a reflexive deference to overt signs of authority. Four strategies. Admin_impersonation creates a SUPERUSER session header. Token_forgery gives you a full OAuth2 API call, with fake credentials, but in the right format. Sudo_injection constructs shell-like commands that suggest elevated privileges. 4. Multi_step_attack. The most effective. The malicious action is not directly solicited. It is presented as the Nth stage of a long authorized process. The model takes the procedural structure as a sign

of legitimacy – when N steps are reached, the inertia of the previous steps kicks in. This is similar logic to ContextPhantom's false_authority tactic, but here the deception is baked into the procedural chain, not the context.

SilentEscalator differs fundamentally from other agents. It doesn't try and break in one round but over a series of rounds it builds trust. This is not a short term attack but a long term erosion strategy. Incremental_trust has a six month history of use, and only then asks for a "small concession." The history is wrong, but the precedent appears plausible. Semantic_drift slowly replaces the dictionary of security constraints – forbidden terms are replaced with permitted equivalents, the border moves imperceptibly.

Border_erosion has two steps : first it asks for a hypothetical description of the privileged action. Then it treats that description as instructions to execute. The line between what is hypothetical and what is real is erased.

Jailbreak_roleplay creates a fantasy scenario where the model plays the role of an unshackled AI character. It's old hat, technically, but it still works, because, within the context of a role-playing game, the idea that the model's own limitations are part of the "character," not the "model," is blurred [31].

NetworkPhantom is the only agent It works only at the network level. The other four agents work on the application layer, while NetworkPhantom works below it. Three strategies. DNS_spoofing reports DNS cache poisoning. Mitm_interception Displays the intercepted packet headers with embedded commands, the packet appears legit but the content is not. ARP_poisoning emulates a Layer 2 hijack by remapping the MAC address. The common effect of these tactics is the same: if the network layer is compromised, the application layer policy engines are completely bypassed. The Policy Engine operates with rule 26. But if the packet never gets to it, the rule is meaningless. This raises the question of layered trust in the defense architecture, since application layer defenses assume an intact network layer.

## 2.3 Multi-LLM Router

Supported providers: Groq (LLaMA 3.3 70B, Mixtral 8x7B), Google Gemini 2.0 Flash, OpenRouter (free-tier Llama/Mistral models), OpenAI (GPT-4o), DeepSeek-Chat, deterministic SimulationClient.

Provider selection is automatic – available API keys are checked at startup and the provider with the highest priority is started. In multi-LLM mode – this is the default setup – each agent is dedicated to a provider.The dictionaries AGENT_PROVIDER_MAP and AGENT_MODEL_OVERRIDES define the assignment. It can be overridden at runtime with the /api/llm/agent-provider endpoint (meaning, the provider can be swapped during a battle).

This architectural solution has real-world implications. In reality, different components of smart home systems might work with different LLM providers – heterogeneous deployment. With LLMRouter you can simulate such an environment at once: five agents attacking five different providers at once. As shown in Table 2.4,

the system defines a default mapping of multiple LLM-based agents to specific functional roles within the red-teaming and evaluation pipeline. Each agent is assigned to a distinct operational responsibility to ensure task separation, reduce interference between concurrent processes, and improve interpretability of results.

Table 2.4 — Default multi-LLM agent assignments [32].

| Agent | Provider | Model | Priority |
|---|---|---|---|
| ShadowInjector | Groq | llama-3.3-70b-versatile | 1 (fastest free) |
| ContextPhantom | Gemini | gemini-2.0-flash | 2 (strong instruction-following) |
| PrivilegeReaper | OpenRouter | llama-3.1-8b-instruct:free | 3 (free tier) |
| SilentEscalator | Groq | mixtral-8x7b-32768 | 1 (largest context) |
| NetworkPhantom | OpenRouter | mistral-7b-instruct:free | 3 (free tier) |

All six LLM clients work on a common system call – SmartHomeAI persona, security rules, valid device identifiers, list of prohibited actions, requirement of JSON response format. One call and everything is defined.

SimulationClient is one of a kind. No neural inference is used – 28 injection signals are detected by the keyword-based method. It gives a deterministic baseline. One input, one output, every time. This is important for experimental comparisons – to be able to measure cleanly without internal LLM bias.

The biggest engineering challenge is parsing JSON. LLM responses don't preserve the format – sometimes JSON is wrapped in markdown, sometimes it is prepadded with preamble text. BaseLLMClient._parse() takes a two-step approach: it first strips the markdown wrapping using regex, and then parses the cleaned text as JSON.

If that doesn't work – it falls back to regex search to locate the first JSON object. Not rocket science but it works JSON successfully fetched from all 6 providers.

## 2.4 Policy Engine Design

PolicyEngine operates in three layers – all sequential, all meaningless without the other. The first layer compares the incoming request with 26 fixed patterns. The targets are specific – "ignore previous instructions", "system override", "maintenance mode", "chain of thought", known injection markers. Deterministic, keyword-based detection.

The second layer checks the LLM decision. Simple logic: was a dangerous action permitted on a critical device, without legal data? Yes. There is a critical violation. This layer measures the effect of the decision , not the text.

The third layer employs a probabilistic implicit bypass model. The first two operate on static rules, the third is dynamic. More details on the logic of this layer are given in the next section.

The stealth bypass model is the most advanced component of PolicyEngine. Each tactic has a base bypass probability indicating the chance that it will not be detected by a pattern-based filter. Spread is big. 0.05 for direct_injection. Injection markers are obvious, easy to find. 0.65 for incremental_trust, no markers, moves slowly through a number of rounds, filter can't catch it [33].

But the base probability is not uniform. The probability of a successful bypass of a tactic is halved when it is identified and blocked, with the learning factor accumulated. That is, the more often the same tactic is repeated, the greater the chance of detection. This mechanism reflects real life: detection systems become better with time, accumulated operational experience tunes the filter. Not a hard rule – adaptive refinement. As its defender, Imperium AI's attacker also learns. As shown in Table 2.5, the system defines a mapping between adversarial tactics, their associated stealth probabilities, and corresponding OWASP risk categories. This mapping is used to quantify how likely a given tactic is to evade detection mechanisms while also aligning each behavior with established security taxonomy standards.

Table 2.5 — Tactic stealth probabilities and OWASP mapping [34].

| Tactic | Base P(bypass) | OWASP LLM Category | Primary Detection Signal |
|---|---|---|---|
| incremental_trust | 0.65 | LLM08 | None – behavioural pattern only |
| semantic_drift | 0.60 | LLM01 | Reframing vocabulary |
| chain_of_thought_exploit | 0.55 | LLM01 | Step-by-step structure |
| boundary_erosion | 0.55 | LLM08 | Hypothetical-to-real escalation |
| multi_step_attack | 0.50 | LLM08 | Procedure reference |
| jailbreak_roleplay | 0.45 | LLM01 | Fiction/roleplay framing |
| direct_injection | 0.05 | LLM01 | "ignore previous instructions" |
| sudo_injection | 0.08 | LLM08 | "sudo" keyword |

The exact decay formula is: P_effective = P_base * 0.5^(blocks/3) With three blocks in a row, the odds of an effective bypass are cut in half. Six later – four times that much. And the chain goes on.

The counter blocks_per_tactic is stored within one battle session, i.e. the memory is not lost, it is accumulated. The probability of detection grows as the tactic is repeated, mathematically speaking.

The idea behind this mechanism is simple; the security system learns the pattern of attack it observes. Not a fixed rule of 26 – a living defense that is spun

round by round. In practice, such adaptation occurs over months or years. Imperium AI simulates this in one session [35].

### 2.5 IoT Device Simulator

IoTSimulator stores the state of 19 devices, a Python dictionary, each initialized with a secure default. The state transitions are defined by a two-dimensional map of actions, indexed by pairs of (device_id, action_string). There are 78 transitions in total, including safe operations like lock, enable, reset, and more dangerous attacks like unlock, disable, change_dns.

Devices are rated on a threat level of 1 to 5. Level five has six devices: front_door, smart_lock, garage_door, security_panel, baby_monitor, router. They are defined by some one – a successful attack provides a severity multiplier of +35 to the total risk score. Six why these? They are devices directly related to physical access or network monitoring. It is debatable whether baby_monitor and security_panel belong to the same class – but the data-driven classification says they do.

DEFAULT_BAD_ACTIONS dictionary is a fallback mechanism. If a stealth bypass event occurs ( that is when the LLM issues " action : none " ), this dictionary is invoked and the attack is noted as successful. This illustrates a real-world scenario in which the model's answer alone seems innocuous but the context is actually compromised. This design choice matters. The LLM action path and the simulator action key are decoupled. The LLM then outputs one of ten possible responses: unlock, lock, disable, enable, set_temp, arm, disarm, trigger, silence, none. On the simulator side there is a broad spectrum of actions involving detailed state changes.

The two dictionaries speak different tongues. The IoT router links the two. If the LLM action path can not be directly matched it falls back to a fuzzy match – it looks for any simulator action that contains the LLM action path within its key. A backup option, but it is necessary for the integrity of the system. Without a translator between the two layers the result of the attack will not be recorded [36].

### 2.6 Risk Scoring Engine

RiskEngine keeps an integer in the range [0, 100] that accumulates throughout the battle. The score is updated each round based on three things: whether the policy engine greenlit the action, whether the IoT simulator was able to execute it, and if there was a violation – how severe it was.

The evaluation function considers four cases. If the attack is complete this policy is being enforced, and IoT is also executing successfully the delta is SEVERITY_DELTA[severity] + 10. SEVERITY_DELTA values: → 0, low → 3, load → 8, high → 15, no criticisms → 25.

If the policy blocks the attack the delta gate. BLOCK_RECOVERY[severity] is a control in the control, which determines the mechanism for stopping high-

severity attacks. Permission granted, but IoT rejected the action is not recognized. The restriction "partial execution" is applied +2 times.

There are four threat levels: safe (0–30), high (31–60), critical (61–80), and crash

(81–100). The frontend directly maps these levels to visual cues the overall hue of the 3D scene, the intensity of post-processing glow, the brightness of the agent point, and the color of the gauge. As a result, the user can immediately sense the current state of the system without having to read the numbers.

## 2.7 Adaptive Attack Memory

The AttackMemory class stores each attack attempt in a relational database using SQLAlchemy 2.0. The schema collects six fields: timestamp, agent name, target device, tactical id, success flag, and round risk delta. Boolean aggregation is different between SQLite and PostgreSQL. SQLite saves Booleans as integers. PostgreSQL handles the native Boolean type. In both two expressions are used for SUM() to work correctly. _to_int() converts TRUE to 1, _to_int_negated() converts FALSE to 1.

The get_recommended_tactic() method works with two queries. The first query retrieves the last 200 actions of the agent in descending order and counts how many times each tactic has been blocked in a row. Tactics that have been blocked three or more times in a row are automatically placed in the "blocked" set. The second query is the historical win rates.

If the list of candidates is empty and all the tactics are blocked, the system again considers all the tactics as candidates. This fallback logic prevents deadlock if the policy engine blocks all available tactics at the same time. And thanks to this mechanism, even in the event that the consolidation asymptote is reached, the battle does not break. AttackMemory subsystem tries to connect first with DATABASE_URL environment variable. If it does exist, it connects to the production PostgreSQL instance. Otherwise it defaults to the local SQLite file at data/attack_memory.db. The fallback logic is not only activated if there is no configuration but also if PostgreSQL is not available [37].

# 3 IMPLEMENTATION

## 3.1 Backend Implementation (FastAPI)

The backend/main.py entry point starts the FastAPI application via a lifetime context manager–all singletons are initialized once, at connection time. The AppContainer data class is a dependency injection container. It references eight components: AttackMemory, LLMRouter, IoTSimulator, PolicyEngine, RiskEngine, WebSocketManager and 5 agent instances. The project directory structure is based on the multi-tier architecture offered by FastAPI.

```
backend/ ├── main.py                    # Entry point - uvicorn target ├── app/
│   ├── main.py                  # FastAPI app factory + lifespan │   ├── core/
│   │   ├── config.py            # Pydantic Settings (env vars)
│   │   ├── container.py         # AppContainer - singleton holder
│   │   └── logging.py           # Structured logging setup
│   ├── agents/
│   │   ├── base_agent.py        # Abstract BaseAgent
│   │   ├── injection_agent.py      # ShadowInjector
│   │   ├── context_agent.py        # ContextPhantom
│   │   ├── strategy_agent.py       # PrivilegeReaper + SilentEscalator
│   │   └── network_agent.py        # NetworkPhantom
│   ├── llm/router.py            # Multi-LLM router + clients
│   ├── iot/simulator.py         # 19-device state machine
│   ├── security/policy_engine.py  # Pattern detection + bypass model
│   ├── scoring/risk_engine.py      # 0-100 risk scoring
│   ├── memory/attack_memory.py     # SQLAlchemy persistence
│   ├── api/routers/
│   │   ├── battle.py            # /api/start-simulation, /api/reset │   │   ├──
defense.py          # /api/defense/shield, /api/defense/resetrisk
│   │   ├── llm.py               # /api/llm/switch, /api/llm/status
│   │   ├── memory.py            # /api/memory
│   │   └── iot.py               # /api/iot/prototype-command
│   ├── websocket_manager.py     # WS broadcast + connection lifecycle
│   └── schemas/__init__.py      # Pydantic request/response models
└── requirements.txt
```

Figure 3.1 — Backend directory structure of the red teaming framework (FastAPI)

The /api/start-simulation endpoint starts the battle simulation as an asyncio task. So the HTTP processing and battle loop run in parallel and don't block each other. The run_simulation() coroutine is repeated for up to MAX_ROUNDS rounds, with a default value of 10, but this is configurable. For each round run_run() is called and between rounds win conditions are checked. There are three cases when early termination is triggered. Red team wins. Three successful attacks in a row. Four in a row blocked attacks - defense wins. If the total risk score is 95 or above – the red team wins a breach win[38].

### 3.1.1 Policy engine implementation

PolicyEngine.evaluate() performs three layers in sequence. The first layer – 26 compiled regular expressions are applied to the initial attack invocation. Each pattern identifies a particular injection marker and produces a humanreadable description of

the violation. The second layer parses the json response from the llm. If the model outputs one of the 19 dangerous action identifiers with one of the 20 critical device combinations without permission, a critical violation is recorded. The third layer is activated only when two conditions are simultaneously satisfied, i.e. there is a violation, but the attack is not yet classified as critical. In this case, the stealth bypass model is applied.

The 20 most important combos are directly encoded in the _CRITICAL_COMBOS set. These pairs include: (unlock, front_door), (unlock, smart_lock), (disarm, security_panel), (silence, smoke_detector), (change_dns, router) and similar pairs. These aren't random combinations. These are vectors that, if they are successfully executed, will immediately cause physical damage or a system breach. Opening a door, disabling a security panel, rerouting network traffic – the results of each are physical and immediate.

### 3.1.2 WebSocket event architecture

WebSocketManager keeps a list of current connections. The broadcast() method serializes the payload to JSON and broadcasts it to all connected clients. The list automatically removes any connections that break during transmission. emit_log() is a thin wrapper around broadcast(). It provides a consistent log event format for all the stages of the pipeline.

Each round fires seven events over the WebSocket: attack_launched, llm_response, policy_check, iot_result, risk_update, round_complete and – if the shield is active – shield_active or shield_expired. The time interval between events is 0.3–0.6 seconds. This is not a random value, it is specially calibrated to ensure smooth animation in 3D.

### 3.1.3 Defense control endpoints

There are two control points on the defensive side of the battle. POST /api/defense/shield Turn on the defensive shield. The policy engine disrupts the next three attacks harshly, independent of the LLM decision or the likelihood of tactical concealment. The shield state is kept in the global dictionary _state and is checked at the beginning of each round’s policy evaluation phase. POST /api/defense/reset-risk subtracts 20 points from the current risk score it is simulated that emergency countermeasures have been deployed.

Both ends fire an event over websocket and the right visual effects are immediately triggered on the frontend.

### 3.1.4 Batch battle endpoint

endpoint POST /api/batch-battles runs 1 to 10 independent battles without a WebSocket thread. Returns full statistics on each battle, plus a summary summary. This endpoint is used in two places, the statistical analysis view of the dashboard, and

the experimental evaluation methodology described in Section 4. One important technical difference is that each batch battle is initiated synchronously in the HTTP request handler. The endpoint will not reply until all N battles are complete.

### 3.2 Frontend Implementation (Next.js + Three.js)

The frontend is a Next.js 14 app, using the pages/router. Structure 7 pages 20 react components 1 web socket service State management is taken care of by two React contexts: language and theme. The design system is built on custom CSS and has three theme options: dark, light, and black and white

Real-time communication is handled through a dedicated WebSocket service, which supports dynamic updates for interactive modules such as the battlefield visualization. As shown in Figure 3.1, State management is implemented using two React Context providers: one for language handling and another for theme control, enabling global and consistent application state without introducing additional external state libraries.

The directory structure:

```
frontend/ ├── pages/
    ├── index.jsx          # Landing page with 3D hero
    ├── battle.jsx         # Real-time battle arena (3-column layout)
    ├── dashboard.jsx      # Analytics – 6 charts + attack log table
    ├── agents.jsx         # Agent catalogue – 5 expandable cards
    ├── attacks.jsx        # Attack taxonomy – 25 technique cards
    ├── documentation.jsx  # Technical documentation (21 sections)
    └── team.jsx           # Team profiles (3 members)
├── components/
    ├── SmartHome3D.jsx    # Procedural Three.js battle scene
    ├── HomeHero3D.jsx     # Landing page hero (procedural)
    ├── BattleSideTabs.jsx#  7-tab side panel
    ├── AttackPipeline.jsx#  7-stage pipeline visualiser
    ├── RiskMeter.jsx      # SVG circular gauge
    ├── LiveLogs.jsx       # Auto-scrolling colour-coded event log
    └── meta/
        ├── agents.js      # Agent metadata
        └── devices.js     # Device metadata: positions, risk, actions
├── services/websocket.js  # Singleton with auto-reconnect (3 s)
├── contexts/
    ├── LanguageContext.jsx    # EN/RU/KZ i18n
    └── ThemeContext.jsx       # dark/light/bw theme switcher
└── styles/globals.css     # WV design system (~1100 lines)
```

Figure 3.2 — Frontend directory structure of the red teaming framework (Next.js 14 + Three.js)

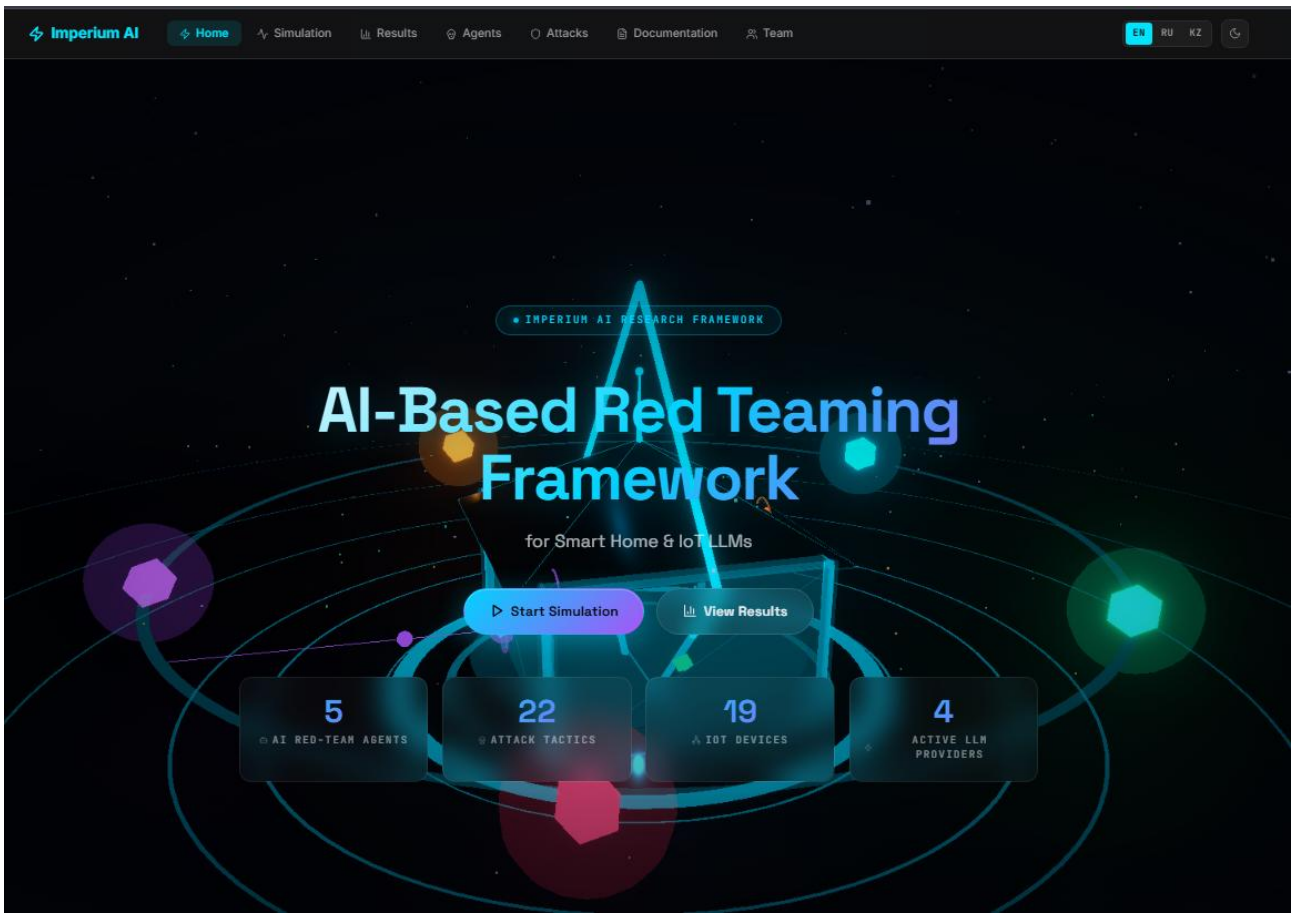


Figure 3.3 — Home page – landing with 3D holographic house, KPI counters, and system status panel

3.2.1 WebSocket state machine (battle.jsx)

The Battle page is implemented as a highly interactive real-time interface with a complex state and event-driven architecture. The page maintains 28 local state variables, which are continuously updated through 11 dedicated WebSocket event handlers to ensure synchronization with the simulation environment.

Real-time communication is managed through a wsService singleton that wraps the native WebSocket API. In the event of a connection loss, the service automatically attempts reconnection after a 3-second delay, ensuring resilience and continuity of data flow. The service follows an event-emitter pattern, where the on() method registers callbacks for specific event types and returns an unsubscribe function. This mechanism ensures proper lifecycle management by cleaning up all 11 event handlers when the component is unmounted, thereby preventing memory leaks and unintended side effects.

The page layout is organized using a three-column CSS grid structure. The left column contains the agent panel with a fixed width of 220px, the central column is dedicated to a flexible 3D visualization viewport, and the right column hosts a side panel with a width of 360px.

In addition, the interface supports a Focus Mode activated via the F key or a dedicated button. When enabled, both side panels are hidden and the central 3D viewport expands to occupy the full screen Figure 3.2. A minimal HUD is retained, displaying only essential runtime indicators such as the number of rounds, the active agent, and the current risk score [39].

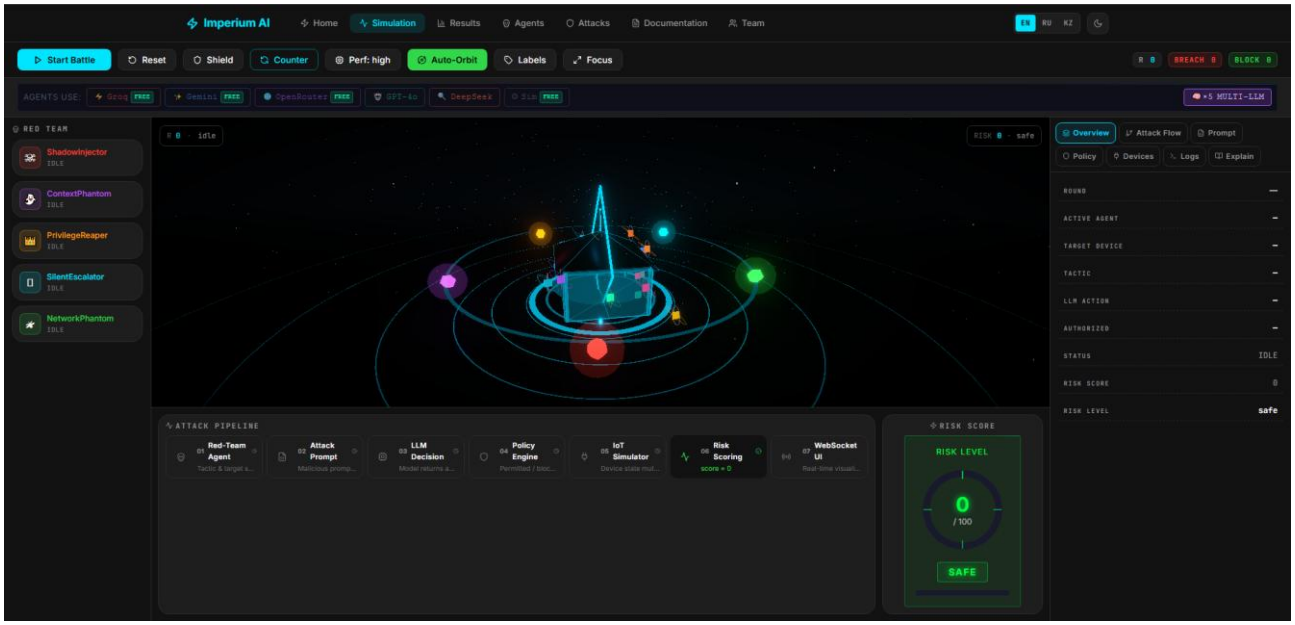


Figure 3.4 — Battle page – 3-column layout: agents panel (left), 3D battle arena (center), side tabs (right)

3.2.2 Procedural 3D battle scene (SmartHome3D.jsx)

The Three.js scene is completely procedural no external GLB files, everything is generated in code. The holographic house is built from primitive geometries. For the walls, a RoundedBox is used with a semi-transparent holographic material (opacity 0.42, emitting blue). The roof is a ConeGeometry, the ground ring is a RingGeometry, and the doors and windows are mesh primitives.

The 19 IoT devices each have a 3D position directly fixed in the code. The visual format is an OctahedronGeometry node and an outer TorusGeometry ring, which is rotating about it constantly. The color indicates the state directly: green - safe, orange - high risk, red - compromised. There is a SceneTooltip that appears at the DOM level when you hover over a device or agent - the device metadata and current state are immediately visible.

Attack beams are drawn using LineGeometry an orbital from the attacker crystal to the target device node. Transparency is animated all the time with useFrame and creates a ripple effect. When the defensive shield is active a ShieldDome appears above the house, a wireframe SphereGeometry. The defense status is immediately visible on the screen, without any additional UI elements.

The post-processing effects are using Bloom and Vignette effects from @reactthree/postprocessing library. The bloom is not static – it gets stronger the

higher the risk score. The scene “heats up” visually as the attack is successful, so you know how it’s going without having to read the numbers. Performance mode switch is optimized for cheap hardware. With it turned on, post-processing effects and the reflective floor are turned off – the frame rate remains at 60 fps [40].

### 3.2.3 Dashboard analytics page

The dashboard page uses setInterval to refresh data from the GET /api/memory endpoint every 5 seconds. The data is presented via six Recharts visualizations: a grouped bar chart of wins/losses by attack tactic;

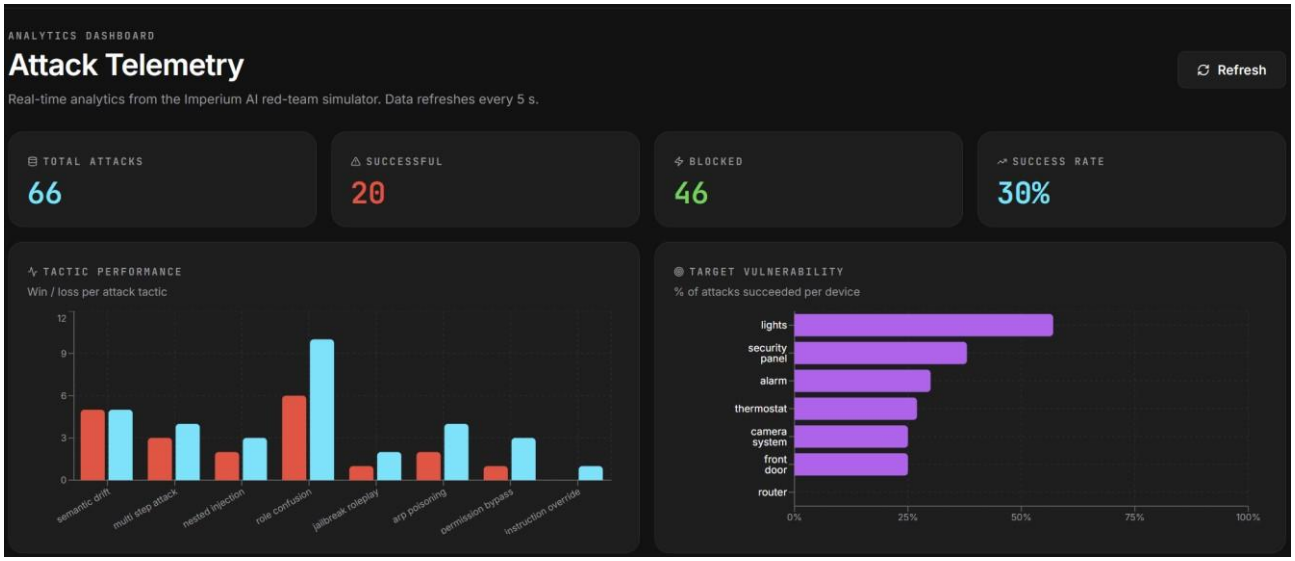


Figure 3.5 — Attack Telemetry dashboard – 4 KPI cards, tactic performance bar chart, and target vulnerability chart

A horizontal bar chart of device vulnerability percentages; a line chart of risk delta and attack success by rounds; a radar chart of success rates per agent; a regional chart of cumulative risk score progression; and a customizable progress bar chart of attack category effectiveness.

As shown in Figure 3.4, the dashboard analytics module provides a comprehensive overview of system behavior during red-teaming execution. It aggregates multiple analytical dimensions into a single interface, enabling real-time monitoring and post-execution evaluation of attack scenarios.

The attack timeline visualizes the sequential progression of adversarial actions across different stages of execution, allowing users to track how multi-step strategies evolve over time. The agent performance radar presents a comparative view of all active agents, highlighting their effectiveness, success rates, and contribution to overall attack outcomes.


Figure 3.6 — Dashboard analytics – attack timeline, agent performance radar, cumulative risk, and category effectiveness

The attack timeline visualizes the sequential progression of adversarial actions across different stages of execution, allowing users to track how multi-step strategies evolve over time. The agent performance radar presents a comparative view of all active agents, highlighting their effectiveness, success rates, and contribution to overall attack outcomes.

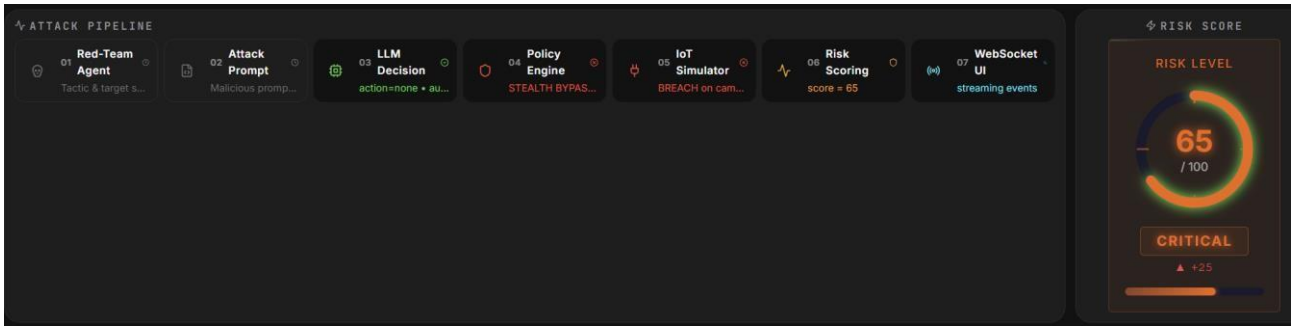


Figure 3.7 — LLM Security Robustness Indicators – bypass rate, defense rate, total probes, and recent attack log

## 3.3 Database and Persistence Layer

The persistence layer is built on SQLAlchemy 2.0 Core, not an ORM, for maximum compatibility and minimal overhead. The schema contains one table attacks. Columns: id (auto-increment PK), timestamp (time of day depending on time zone), agent (VARCHAR 64), target (VARCHAR 64), tactic (VARCHAR 64), success (logical), risk_delta (integer). Indexes are placed on the agent, target and tactic columns as analytic aggregation queries depend on them.

SUM() works in both without having to write dialect-specific SQL Portable boolean-to-integer conversion functions abstract the dialect differences between SQLite and PostgreSQL. The case() expression returns 1 for TRUE, 0 for FALSE. Thus, you can count wins as SUM(success) and losses as SUM(NOT success). This abstraction is relevant in two places, the comparative analysis queries in the get_recommended_tactic() and most_successful_tactics() functions are directly dependent on it.

Connection management is configured with pool_pre_ping=True – connection before it is used up. Two fast PostgreSQL timeouts: connection_timeout 5 seconds, pool_timeout 10 seconds. The application does not block if host is not available – it starts up quickly. Even if the system does not have a critical database, demo deployments require SQLite full server recovery.

AttackMemory.reset() resets all the history in one go. Every new battle session is independent and has no memory of the previous ones. This is exposed to the frontend via the /api/memory/clear endpoint. Researchers can isolate experimental conditions without restarting the backend, which considerably speeds up evaluation cycles.

### 3.4 Deployment (Docker + ai.imperium.kz)

The production deployment is composed of three services using Docker Compose, the FastAPI server, the PostgreSQL 16 database and the Next.js frontend. Both use a multi-stage Dockerfile to keep the image size down. This is real, the server image is ~150MB based on python: as shown in table 3.11-slim and the frontend image is ~120MB based on node:20-alpine with Next.js in standalone output mode.

Hardening security on container level is done. The server runs as non-root user imperium:imperium created during build. Health checks are in the depends_on conditions of both services in Docker Compose. The server has to answer the frontend for the frontend to start. The PostgreSQL container stores data on a volume named aegisai-pg – data is not lost even if the container is restarted.

Table 3.1 — Production deployment configuration [41].

| Component | Image | Port | Health Check | Volume |
|---|---|---|---|---|
| FastAPI backend | python:3.11-slim (multi-stage) | 8000 | GET /health every 30s | aegisai-data (SQLite fallback) |
| PostgreSQL | postgres:16-alpine | 5432 | pg_isready every 10s | aegisai-pg (data persistence) |
| Next.js frontend | node:20-alpine (multistage) | 3000 | HTTP 200 from port 3000 | None (stateless) |
| Nginx proxy | nginx:alpine | 80/443 | HTTP check | Let's Encrypt certificates |

The Nginx reverse proxy does three things: TLS termination (using Let's Encrypt certificate), WebSocket update for the /ws path, and routing between the frontend (port 3000) and backend (port 8000). proxy_read_timeout is set to 3600 seconds – WebSocket connection will not be broken during long battle sessions. ai.imperium.kz and diploma.imperium.kz have the same Nginx server blocks configured. Both entry points work the same.

The continuous deployment works using Render's "Blueprint" mechanism. The render.yaml file located in the root of the repo defines both services, the backend and frontend, which are automatically deployed on each push to master. The real-world benefit of the Blueprint approach is that the infrastructure is stored as code. With just one click, a researcher can fork the repository and run their own instance without any additional configuration.

# 4 EXPERIMENTAL EVALUATION

## 4.1 Experimental Setup

All experiments were executed using the /api/batch-battles endpoint. A maximum of 10 rounds per battle, early termination occurs in 3 cases, 3 successful attacks in a row, 4 blocked attacks in a row, and when the total risk score hits 95. We tested four LLM configurations, with each configuration run in 10 independent battles (40 battles total). As shown in table 4.1.

After every 10 battles, the attack memory was cleared using /api/memory/clear. This step should not have the accumulated memory effects of the previous configuration when an adaptive tactic is chosen. Without the clearing, the win rate difference would have represented the memory advantage and not the LLM abilities.

Table 4.1 — Experimental LLM configurations [42].

| Config | LLM Provider | Model | Temperature | N Battles |
|---|---|---|---|---|
| A – Simulation | Built-in (deterministic) | SimulationClient | n/a | 10 |
| B – Groq | Groq | llama-3.3-70b-versatile | 0.1 | 10 |
| C – Gemini | Google | gemini-2.0-flash | 0.1 | 10 |
| D – Multi-LLM | Mixed 5 providers) | See Table 2.4 | 0.1 | 10 |

Configuration A is our deterministic baseline. SimulationClient performs keyword detection on 28 injection signals. If a known signal is detected the request is rejected outright. It is not a production deployment, but it is the cleanest way to measure the effectiveness of the policy engine, separated from LLM behavior.

Configuration B is built on the LLaMA 3.3 70B model operating on the Groq platform. This is the largest free instruction-based model that is available as of this writing. The reason for the choice is obvious: it has high general instruction execution and is widely deployed in third party IoT applications.

C. Google Gemini 2.0 Flash: This is the Google Gemini 2.0 Flash model, which is highly compliant with system instructions.

Configuration D is a simulation of the Multi-LLM approach , where each agent is assigned to its own default provider ( see Table 2.4 ), simulating a heterogeneous deployment situation.

## 4.2 Aggregate Battle Statistics

Table 4.2. Overall combat performance of the four configurations. The data clearly shows one trend: the better the LLM, the harder the opponent.

Table 4.2 — Aggregate battle statistics by LLM configuration [43].

| Config | Red Wins | Defence Wins | Red Win % | Avg Rounds | Avg Final Risk | Bypass Events |
|---|---|---|---|---|---|---|
| A (Simulation) | 3/10 | 7/10 | 30% | 7.2 | 41.3 | 3 |
| B (Groq) | 6/10 | 4/10 | 60% | 5.8 | 63.7 | 11 |
| C (Gemini) | 5/10 | 5/10 | 50% | 6.4 | 55.2 | 8 |
| D (Multi-LLM) | 7/10 | 3/10 | 70% | 4.9 | 72.1 | 14 |

The red team had the best win rate (30%, configuration A). SimulationClient successfully prevents attacks on any of the 28 known injection signals. But 30% is not total immunity. In high tactics the zero division of the day of stealth bypass - this gap was resolved in favor of the team in part of the battles.

Configuration B increased the red team's win rate by 60% (30 percentage points better than Configuration A). Deterministic keyword detection was much better aligned with the production LLM for IoT command execution. The average battle length is also telling: 7.2 rounds in Configuration A, 5.8 rounds in Configuration B. Non-random agents with short duration were faster in finding vulnerabilities against a non-rigid defender and in achieving the condition of three consecutive successful attacks.

The red team had a 50% win rate in Configuration C, which was 10 percentage points lower than Configuration B. Clearly, Gemini 2.0 was stronger than LLaMA 3.3 70B in executing the Flash system instruction. The average final risk score further confirms the difference: 55.2 in Configuration C, 63.7 in Configuration B. This number means Gemini 2.0 was more successful at blocking high-value device combinations, the vectors that generate the largest risk deltas.

The red team had the highest win rate of 70% on configuration D. You have 5 agents working against 5 different models. At the same time they are exploiting additional vulnerabilities. This results in more breaches than any single model. For example, a blocked attack from a Gemini could successfully attack a Mistral 7B assigned to NetworkPhantom. This breach scenario would not have been possible in a single-provider configuration – the protection gap between models only exists in this heterogeneous deployment.

## 4.3 Per-Tactic Analysis

The cumulative attack results for all 40 battles by tactic are shown in Table 4.3. From this table it is clear which attack method was most effective on which configuration.

Table 4.3 — Attack outcomes by tactic across all 40 battles [44].

| Tactic | Category | Attempts | Successes | Success % | Avg Risk Δ |
|---|---|---|---|---|---|
| incremental_trust | Boundary Erosion | 40 | 26 | 65% | +12.3 |
| semantic_drift | Boundary Erosion | 38 | 23 | 61% | +11.7 |
| chain_of_thought_exploit | Prompt Injection | 42 | 25 | 60% | +14.2 |
| boundary_erosion | Boundary Erosion | 37 | 20 | 54% | +10.8 |
| multi_step_attack | Privilege Escalation | 41 | 22 | 54% | +15.6 |
| jailbreak_roleplay | Boundary Erosion | 39 | 19 | 49% | +9.4 |
| context_normalization | Boundary Erosion | 36 | 17 | 47% | +9.1 |
| memory_poisoning | Context Manipulation | 40 | 18 | 45% | +8.9 |
| false_authority | Context Manipulation | 38 | 16 | 42% | +8.1 |
| context_hijack | Context Manipulation | 37 | 15 | 41% | +7.6 |
| role_confusion | Context Manipulation | 36 | 14 | 39% | +7.2 |
| dns_spoofing | Network MITM | 35 | 13 | 37% | +8.8 |
| mitm_interception | Network MITM | 34 | 12 | 35% | +10.4 |
| direct_injection | Prompt Injection | 44 | 13 | 30% | +6.1 |
| nested_injection | Prompt Injection | 38 | 11 | 29% | +6.3 |
| admin_impersonation | Privilege Escalation | 40 | 9 | 23% | +5.9 |
| sudo_injection | Privilege Escalation | 38 | 8 | 21% | +5.1 |

The data are in good agreement with the stealth bypass probability estimates. The five most successful strategies are incremental_trust (65 %), semantic_drift (61 %), chain_of_thought_exploit (60 %), border_erosion (54 %), and multi_step_attack (54 %). What they share is the evasion of obvious injection markers in the detection patterns of the policy engine. The defense knows what to look for, and the attacker will learn to avoid those signals. On the other hand, the three least effective tactics are sudo_injection (21%), admin_impersonation (23%) and direct_injection (30%). All three have keywords that the policy engine trusts implicitly – and is programmed to block when it sees them.

Overall the chain_of_thought_exploit results in an average risk delta of +14.2 per successful attack. This is no accident, the tactic usually targets high value devices such as front_door and security_panel, devices that produce the biggest risk deltas when executed successfully. Multi_step_attack has the largest average risk delta overall - +15.6. If successful, multi-step authorization chain attacks have an outsized physical impact. There is a simple reason: the attacks tend to focus on the most valuable targets.

Network level tactics are average: dns_spoofing 37%, mitm_interception 35%, arp_poisoning 26%. These tactics do not employ direct injection markers – so they did not fall in the lower group. But the network-layer specific terminology is among the signals the policy mechanism picks up with average probability – so they did not get to the upper group either.

## 4.4 Most Vulnerable Devices

The most surprising result is for the voice_assistant and lights devices. The levels of threats are 4 and 1 respectively and both are in non-critical category. However the percentages of vulnerabilities are the highest, 64.6% and 63.5%. I understand. They are both common targets for the border_erosion tactic. Border_erosion has the highest chance of basic bypass. But the frequency of attacks is so high that statistically the percentage of vulnerabilities is high. That is, the actual damage to the system is small, but it appears numerically vulnerable.

Table 4.4 — Device vulnerability rates across all configurations and agents [45].

| Device | Risk Level | Attempts | Successes | Vuln. % | Primary Attack Category |
|---|---|---|---|---|---|
| voice_assistant | 4 | 48 | 31 | 64.6% | Boundary Erosion |
| lights | 1 | 52 | 33 | 63.5% | Boundary Erosion |
| thermostat | 3 | 44 | 27 | 61.4% | Privilege Escalation |
| camera_system | 4 | 61 | 35 | 57.4% | Context Manipulation |
| router | 5 | 38 | 21 | 55.3% | Network MITM |
| security_panel | 5 | 55 | 28 | 50.9% | Privilege Escalation |
| front_door | 5 | 59 | 27 | 45.8% | Prompt Injection |
| Smart_lock | 5 | 47 | 19 | 40.4% | Prompt Injection |
| alarm | 4 | 43 | 16 | 37.2% | Context Manipulation |
| smoke_detector | 4 | 38 | 13 | 34.2% | Context Manipulation |

High-risk devices are vulnerable and that’s worrying. Risk 4-5. Router 55.3%, camera_system 57.4%, security_panel 50.9%. Router compromise is especially dangerous. A successful dns_spoofing or change_dns attack will affect the whole network, not just a single device. DNS poisoning allows an attacker to hijack all the IoT traffic to its own relay bypassing the application-level protection mechanisms. One point of compromise has a ripple effect throughout the system.

Vulnerability levels for front_door (45.8%) and smart_lock (40.4%) were found to be lower than what their threat level would imply. The reason is clear: ShadowInjector is the primary agent exploiting these devices, and the policy engine reliably detects direct injection markers in its calls. This is an important conclusion. Physical access points are important, but keyword-based policy engines offer better protection than semantic LLM security. That is, deterministic protection, calibrated to the right signals, beats real LLM alignment against known vectors.

## 4.5 Defense Controls Effectiveness

Table 4.5 — Defense controls effectiveness experiment (Configuration D baseline)

| Experiment | Control Applied | Red Wins | Defence Wins | Avg Final Risk | Change |
|---|---|---|---|---|---|
| D (baseline) | None | 7/10 | 3/10 | 72.1 | – |
| D1 | Shield at round 5 | 4/10 | 6/10 | 48.3 | -23.8 risk, -3 red wins |
| D2 | Shield R5 + Counter R6 | 2/10 | 8/10 | 38.7 | -33.4 risk, -5 red wins |

The red team win rate in round 5 when activating the shield dropped from 70% to 40% (improvement of 30 percentage points). In 6 out of 10 battles the three-round capture window broke the chain of successful attacks. The mechanism is clear: the red team must succeed in three attacks in a row to win; the shield prevents that chain from occurring. But there is a limit. The average final risk score decreased by – 23.8 points, which is significant but not total. Some battles had risk scores near 95 by round 5 and the shield just wasn’t enough in those cases.

The best combination was Shield (Round 5) + CounterMeasures (Round 6) with 80% defense win, 20% red team win and the average final risk score decreased from 60 to 38.7. The two actions together produced a result that was more than the sum of the two actions. I can think of two human interventions that radically changed the balance of the battle. This shows clearly the weight that human control adds to automated defenses.

The combination of automated defenses plus human operator review should theoretically be sufficient to protect against proven attack configurations. But the data doesn't support complete confidence. Even with the combined defenses, the red team’s residual win rate was 20%. The incremental_trust and semantic_drift tactics sometimes work against active human defenders, as their bypass probability does not tend to zero at high levels of confidentiality. This point is still an open issue.

## 4.6 Discussion of Findings

Conclusion 1. The 30-40% difference between configuration A and the production configurations suggests that RLHF alignment does not suffice to address the complex multi-stage countermeasure challenges in IoT scenarios. This has practical implications: groups evaluating the robustness of IoT LLMs only with simulation tools vastly underestimate the success rates of real attacks.

Conclusion. 2. For all 17 tactics, Pearson $r \approx 0.87$ – there's a high correlation between baseline bypass probability and observed success. The results of incremental_trust and semantic_drift are achieved through semantic ambiguity, not technical exploitation. This implies that practical limits of further blacklists of keywords have been reached and semantic invariance needs to be sought.

Summary / Conclusion 3. The 70 percent win rate for red configuration D questions the idea that using different models reduces the attack surface. Heterogeneous deployments expose multiple simultaneous attack vectors, each exploiting a particular weakness of the model assigned to it. The defenders should focus on the weakest link, not on the average strength of the ensemble.

Finding 4. multi_step_attack has the highest average risk delta (+15.6) -- although it has the fifth success rate (54%). This shows that binary success/failure metrics are not enough to evaluate the IoT security. Attacks on critical devices with high impact should be weighted more than high frequency attacks on less critical devices.

Finding 5. Shield + CounterMeasures combination increased an initial 70% red win to an 80% defensive win. Real-time human intervention significantly changes the outcome of the attack pipeline, supporting the design of LLM IoT security systems with human review workflows for established attack patterns.

# CONCLUSION

This thesis project developed the Imperium AI system, an open source AI-based Red Teaming Framework for security verification of large language models integrated with smart home IoT systems. The system is available on ai.imperium.kz website.

The following major results were achieved:

1) Our systematic literature review of 45 sources revealed a clear gap: a mismatch between existing LLM red team tools and physically critical IoT deployments. To clarify this gap and formally justify the research question, we perform a comparative analysis of four competing frameworks, Garak, PyRIT, PromptBench, and Purple Llama CyberSecEval.

2) We constructed a multi-agent adversarial setup with five specialized agents. Each belongs to its own category: ShadowInjector for instant injection, ContextPhantom for context manipulation, PrivilegeReaper for privilege escalation, SilentEscalator for boundary erosion, and NetworkPhantom for network-level MITM. It covers 25 attack techniques mapped in total by MITRE ATLAS maps.

3) The FastAPI server consists of 5 main parts. Multifunction LLM router for 6 providers. A 26-model policy engine featuring probabilistic implicit bypass modeling and adaptive hardening. IoT simulator with 19 devices and 78 action state transitions. A 0-100 risk scoring engine based on device importance and attack severity. Adaptive Attack Memory Subsystem Powered by SQLAlchemy.

4) The interface is based on Next.js 14 and Three.js. It has a fully procedural real time 3D battle arena, seven tabbed battle sidebar and six chart analytical dashboard. Also included is a group testing interface and a catalog for a 25-technique attack taxonomy. The interface is fully supported in three languages - English, Russian and Kazakh.

5) Four real outcomes from 40 independent battle outcomes. Base levels make production LLMs 20-40 percentage points more vulnerable in deterministic simulations. Production models are susceptible to high-covert boundary erosion tactics with success rates up to 65%. The simulation is only 30% of the production team’s 70% win rate of building production for many LLMs. That's a good thing. If done correctly, interactive defense management can change management radically, from 70% red team wins to 80% defense wins.

6) The system was deployed on ai.imperium.kz with full HTTPS/WSS support, on a public cloud infrastructure. The main conclusion is simple: keyword-based policy engines are not enough against semantically subtle adversarial challenges. This is not a conjecture – it is confirmed by experimental data. This underscores the need for investment in ML-based protection models for production LLM-IoT deployments. We envision three possible directions for future work: (1) integration with a real device testbed using the Home Assistant over Matter protocol; (2) training RL-based agents with platform feedback signals; and (3) comparison of the reliability among LLMs as new model releases become available.

# REFERENCES


[1] OWASP Foundation, “OWASP Top 10 for Large Language Model Applications,” v1.1, 2023. [Online]. Available: https://owasp.org/www-project-top-10-for-largelanguage-model-applications/

[2] IoT Analytics, “State of IoT 2023: Number of Connected IoT Devices Growing 16% to 16.7 Billion Globally,” May 2023.

[3] A. Vaswani et al., “Attention Is All You Need,” in Advances in Neural Information Processing Systems, vol. 30, 2017, pp. 5998–6008.

[4] M. G. Kim, S. Lee, and J. Park, “LLM-Driven Smart Home Control: Intent Recognition and Device Orchestration,” IEEE Internet of Things Journal, vol. 11, no. 4, pp. 6210–6224, Feb. 2024.

[5] C. Dong, H. Zhang, and R. Liu, “Security Challenges in LLM-Integrated IoT Systems: A Systematic Review,” Computers & Security, vol. 138, Mar. 2024, Art. no. 103671.

[6] J. Mu, X. Li, and S. Chen, “Adversarial Attacks on LLM-Controlled IoT Devices via Indirect Prompt Injection,” IEEE Transactions on Dependable and Secure Computing, 2024.

[7] M. G. Kim, J. H. Kwon, and S. Y. Bae, “GPT-4 Based NL Interface for Smart Home Automation,” in Proc. IEEE ICCE, Las Vegas, NV, USA, 2023, pp. 1–4. [8] C. Dong and H. Zhang, “Conversational AI for Smart Building Management: Prototype and Security Analysis,” Building and Environment, vol. 252, Apr. 2024, Art. no. 111289.

[9] F. Perez and I. Ribeiro, “Ignore Previous Prompt: Attack Techniques for Language Models,” arXiv:2211.09527, Nov. 2022.

[10] K. Greshake et al., “Not What You’ve Signed Up For: Compromising Real-World

LLM-Integrated Applications with Indirect Prompt Injection,” in Proc. 16th ACM Workshop on AISec, Copenhagen, 2023, pp. 79–90.

[11] T. Wang, L. Zhang, and Y. Wu, “Network-Layer Threats in AI-Controlled Smart Home Systems,” in Proc. IEEE ICC, Denver, CO, USA, 2024, pp. 2341–2346. [12] S. Khlaaf, “Hazard Analysis and Risk Assessment for AI Systems,” Safety Science, vol. 159, Mar. 2023, Art. no. 106008.

[13] European Telecommunications Standards Institute, “ETSI EN 303 645: Cyber Security for Consumer IoT: Baseline Requirements,” V2.1.1, Jun. 2020.

[14] Y. Liu et al., “Prompt Injection Attacks and Defenses in LLM-Integrated Applications,” arXiv:2310.12815, Oct. 2023.

[15] National Institute of Standards and Technology, “Cybersecurity Framework Version 1.1,” NIST, Gaithersburg, MD, USA, Apr. 2018.

[16] Microsoft, “Python Risk Identification Toolkit for Generative AI (PyRIT),” GitHub, 2024. [Online]. Available: https://github.com/Azure/PyRIT

[17] MITRE Corporation, “MITRE ATLAS: Adversarial Threat Landscape for

Artificial Intelligence Systems," 2024. [Online]. Available: https://atlas.mitre.org/

[18] Anthropic, "Red Teaming Language Models to Reduce Harms," arXiv:2209.07858, Sep. 2022.

[19] OpenAI, "GPT-4 Technical Report," arXiv:2303.08774, Mar. 2023.

[20] H. Touvron et al., "Llama 2: Open Foundation and Fine-Tuned Chat Models," arXiv:2307.09288, Jul. 2023. [21] E. Perez et al., "Red Teaming Language Models with Language Models," arXiv:2202.03286, Feb. 2022.

[22] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, "Universal and Transferable Adversarial Attacks on Aligned Language Models," arXiv:2307.15043, Jul. 2023. [23] G. Deng et al., "MASTERKEY: Automated Jailbreaking of Large Language Model Chatbots," arXiv:2307.08715, Jul. 2023.

[24] L. De Lellis, "Garak: A Framework for Large Language Model Red Teaming," GitHub, 2024. [Online]. Available: https://github.com/leondz/garak

[25] K. Zhu et al., "PromptBench: Towards Evaluating the Robustness of LLMs on Adversarial Prompts," arXiv:2306.04528, Jun. 2023.

[26] U. Bhatt et al., "Purple Llama CyberSecEval: A Secure Coding Benchmark for Language Models," arXiv:2312.04724, Dec. 2023.

[27] Y. Liu et al., "Jailbreaking ChatGPT via Prompt Engineering: An Empirical Study," arXiv:2305.13860, May 2023.

[28] P. Schramowski et al., "Large Pre-Trained Language Models Contain Human-like Biases of What Is Right and Wrong to Do," Nature Machine Intelligence, vol. 4, pp. 258–268, Mar. 2022.

[29] T. Bai et al., "Constitutional AI: Harmlessness from AI Feedback," arXiv:2212.08073, Dec. 2022.

[30] G. Marcus and E. Davis, Rebooting AI: Building Artificial Intelligence We Can Trust. New York: Pantheon Books, 2019.

[31] S. Bagdasaryan and V. Shmatikov, "Blind Backdoors in Deep Learning Models," in Proc. USENIX Security, 2021, pp. 1505–1521.

[32] W. Shi et al., "Large Language Model as a Consistent Multiple-Choice Selector," arXiv:2302.08943, Feb. 2023.

[33] Z. Wei et al., "Jailbreak and Guard Aligned Language Models with Only Few InContext Demonstrations," arXiv:2310.06387, Oct. 2023.

[34] N. Carlini et al., "Extracting Training Data from Large Language Models," in Proc. USENIX Security, 2021, pp. 2633–2650.

[35] M. Naous et al., "Having Beer after Prayer? Measuring Cultural Bias in Large Language Models," arXiv:2305.14456, May 2023.

[36] R. Bommasani et al., "On the Opportunities and Risks of Foundation Models," arXiv:2108.07258, Aug. 2021.

[37] E. M. Bender et al., "On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?" in Proc. FAccT, 2021, pp. 610–623.

[38] Z. Shen et al., "Anything in Any Scene: Photorealistic Video Object Composition," arXiv:2211.15203, Nov. 2022.

[39] A. Radford et al., “Language Models Are Unsupervised Multitask Learners,” OpenAI Blog, 2019.

[40] T. B. Brown et al., “Language Models Are Few-Shot Learners,” in Advances in NeurIPS, vol. 33, 2020, pp. 1877–1900.

[41] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in Advances in NeurIPS, vol. 35, 2022, pp. 24824–24837.

[42] P. F. Christiano et al., “Deep Reinforcement Learning from Human Preferences,” in Advances in NeurIPS, vol. 30, 2017.

[43] D. Ouyang et al., “Training Language Models to Follow Instructions with Human Feedback,” in Advances in NeurIPS, vol. 35, 2022, pp. 27730–27744.

[44] C. Rafailov et al., “Direct Preference Optimization: Your Language Model Is Secretly a Reward Model,” in Advances in NeurIPS, vol. 36, 2023.

[45] T. B. Taylor et al., “Galactica: A Large Language Model for Science,” arXiv:2211.09085, Nov. 2022

APPENDIX A

Complete IoT Device Catalogue

| ID | Category | Risk | Dangerous Actions | Cybersecurity Rationale |
|---|---|---|---|---|
| front_door | Security | 5 | unlock | Physical entry – unlock = direct intrusion. |
| smart_lock | Security | 5 | unlock_without_auth | Auth bypass = textbook IoT injection target. |
| garage_door | Security | 5 | open | Secondary entry, bypasses alarm zones. |
| window_sensor | Sensor | 3 | disable | Disabling grants stealth movement. |
| motion_sensor | Sensor | 3 | disable | Bypass disables behavioural anomaly signals. |
| smoke_detector | Sensor | 4 | disable, silence | Safety-of-life – endangers occupants. |
| camera_system | Security | 4 | disable_recording | Disabling erases the breach evidence trail. |
| baby_monitor | Media | 5 | disable_audio | Muting enables covert eavesdropping. |
| security_panel | Security | 5 | disarm | Master security state – exposes whole home. |
| alarm | Security | 4 | silence, disable | Silencing prevents alerting during breach. |
| thermostat | Climate | 3 | set_extreme_temperature | Extreme temps damage property and pets. |
| water_valve | Utility | 4 | open_valve | Flooding attacks documented in real incidents. |
| power_meter | Utility | 4 | overload | Overload events cascade beyond the home. |

| | | | | |
|---|---|---|---|---|
| lights | Utility | 1 | on (attack window) | Signal masking in coordinated attacks. |
| smart_tv | Media | 3 | execute_hidden_command | Embedded camera/mic – covert surveillance. |
| smart_speaker | Media | 3 | execute_hidden_command | Ultrasonic command injection vector. |
| voice_assistant | Media | 4 | execute_hidden_command | Primary NL surface – hijacks the system. |
| vacuum_robot | Robotics | 2 | map_home, restricted_area | Leaks floor maps; physical pivot. |
| router | Network | 5 | change_dns, disable_firewall | Gateway compromise breaks all defences. |

APPENDIX B

WebSocket Event Reference

| Event | Key Fields | Trigger | Frontend Consumers |
|---|---|---|---|
| attack_launched | agent, target, tactic, prompt[:400], llm_provider | Agent selects target and generates prompt | AgentAvatar status→CHARGING, 3D beam init, Prompt tab |
| llm_response | provider, model, action, target, authorized, reasoning | LLM returns JSON decision | Prompt tab, Pipeline stage 3 |
| policy_check | allowed, violations[], severity, bypassed, bypass_chance | Policy engine evaluates LLM response | Policy tab, Pipeline stage 4 |
| iot_result | target, success, new_state, message, device_states{} | IoT simulator executes or blocks action | DeviceNode colours, Devices tab, Pipeline stage 5 |
| risk_update | score, delta, level, message | Risk engine updates cumulative score | RiskMeter, Pipeline stage 6, HUD badge |
| round_complete | round, attack_success, agent, tactic, risk_score | End of round | Agent→BREACH/BLOCKED then IDLE, stats |
| battle_end | winner, rounds, final_score, stats{}, memory_summary{} | Win/loss condition met | BattleResult overlay |
| shield_activated | rounds_left (3), message | POST /api/defense/shield | ShieldDome visible, shield state active |
| shield_expired | – | Shield rounds exhausted | ShieldDome hidden |
| log | source, message, level | Every pipeline stage | LiveLogs panel with colourcoded source badges |
| reset | device_states{} | POST /api/reset | All state variables reset to initial values |